Roma Tre

INGEGNERIA
CIVILE, INFORMATICA E DELLE TECNOLOGIE AERONAUTICHE

Information Visualization

Node-Link Visualization of Trees

M. Patrignani

 copyright ©2024 m.patrignani

Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

Overview

- Kind of trees
- Representations of trees
- Node-link representations

 copyright ©2024 m.patrignani

Kinds of trees: rooted or unrooted

- A rooted tree is a tree with a special node that is the root
  - any other node has a parent
- Unrooted trees are connected graphs without cycles
  - no special node is given
  - the tree can be rooted at any vertex to obtain a rooted tree
  - in the following we will only consider rooted trees

 copyright ©2024 m.patrignani

Kinds of trees: ordered or unordered

- An ordered tree is a rooted tree such that the order of the children is semantically significant
  - we cannot permute them
  - example: children in a family tree are ordered based on their age
- An unordered tree allows for permutations of its subtrees
  - example: phylogenetic trees

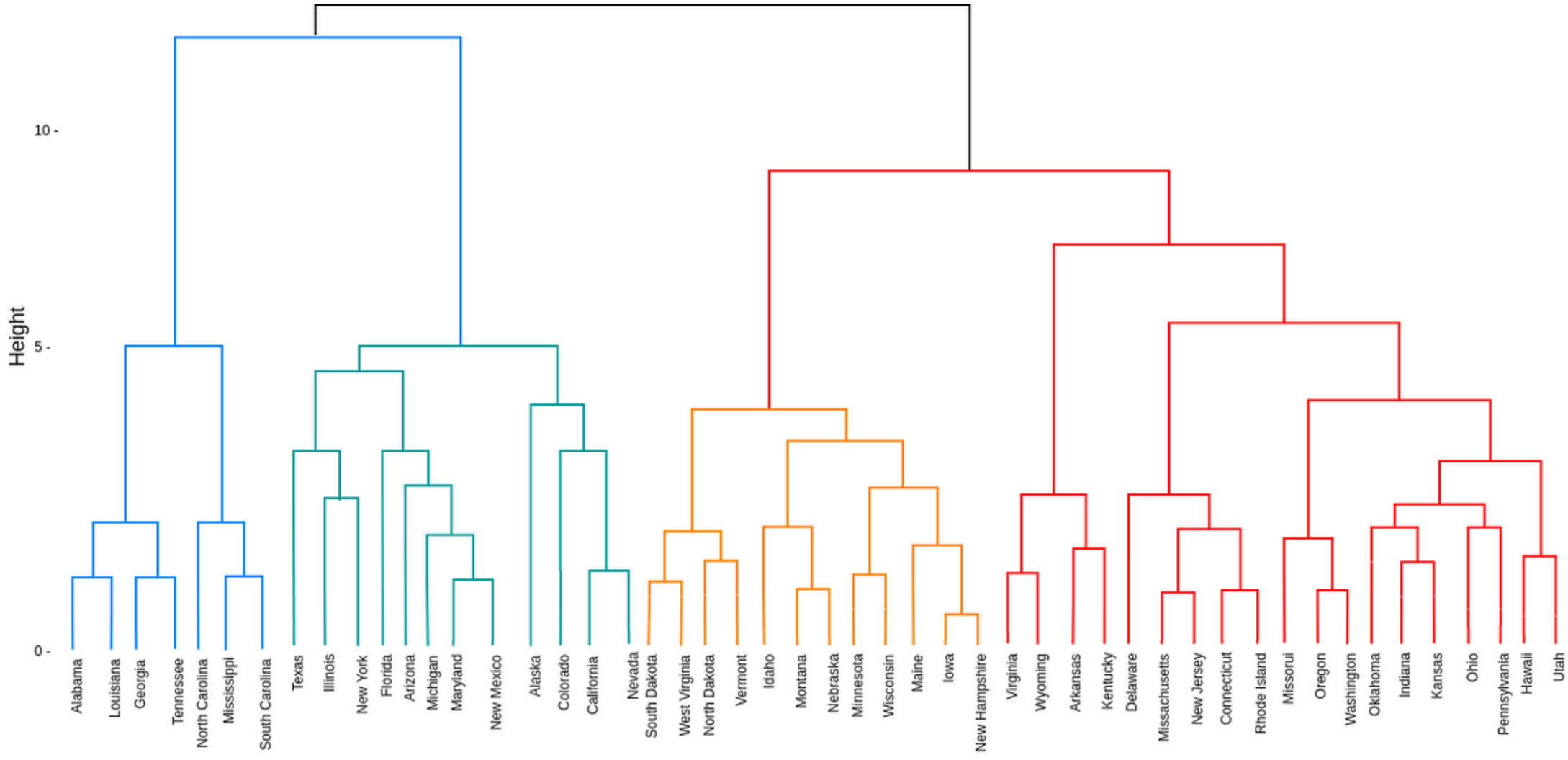
 copyright ©2024 m.patrignani

Kind of trees: binary or arbitrary

- A binary tree is a rooted tree such that each node has at most two children
  - usually called *left child* and *right child,* if the tree is ordered
- In a tree of arbitrary degree a node can have an unbounded (but finite) number of children
- In a k-ary tree a node can have at most k children

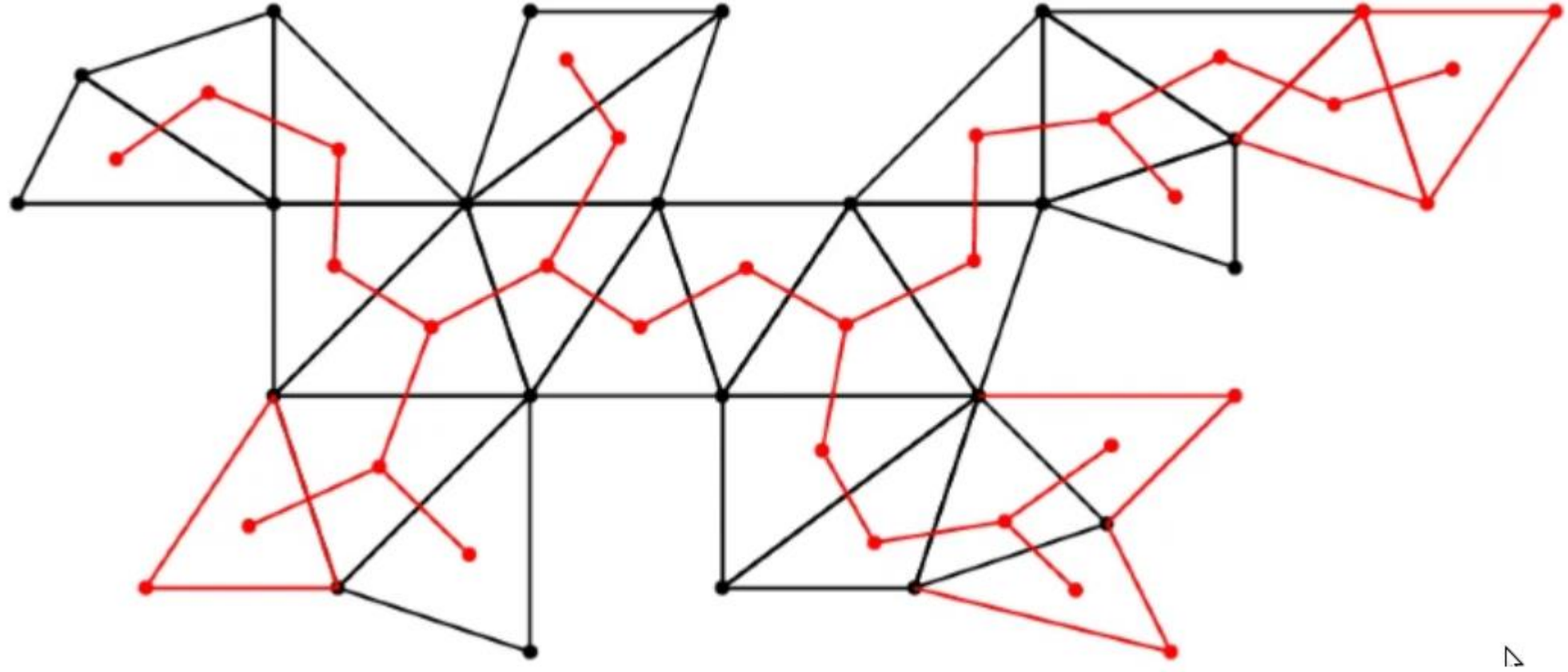
 copyright ©2024 m.patrignani

Example: cluster dendrogram



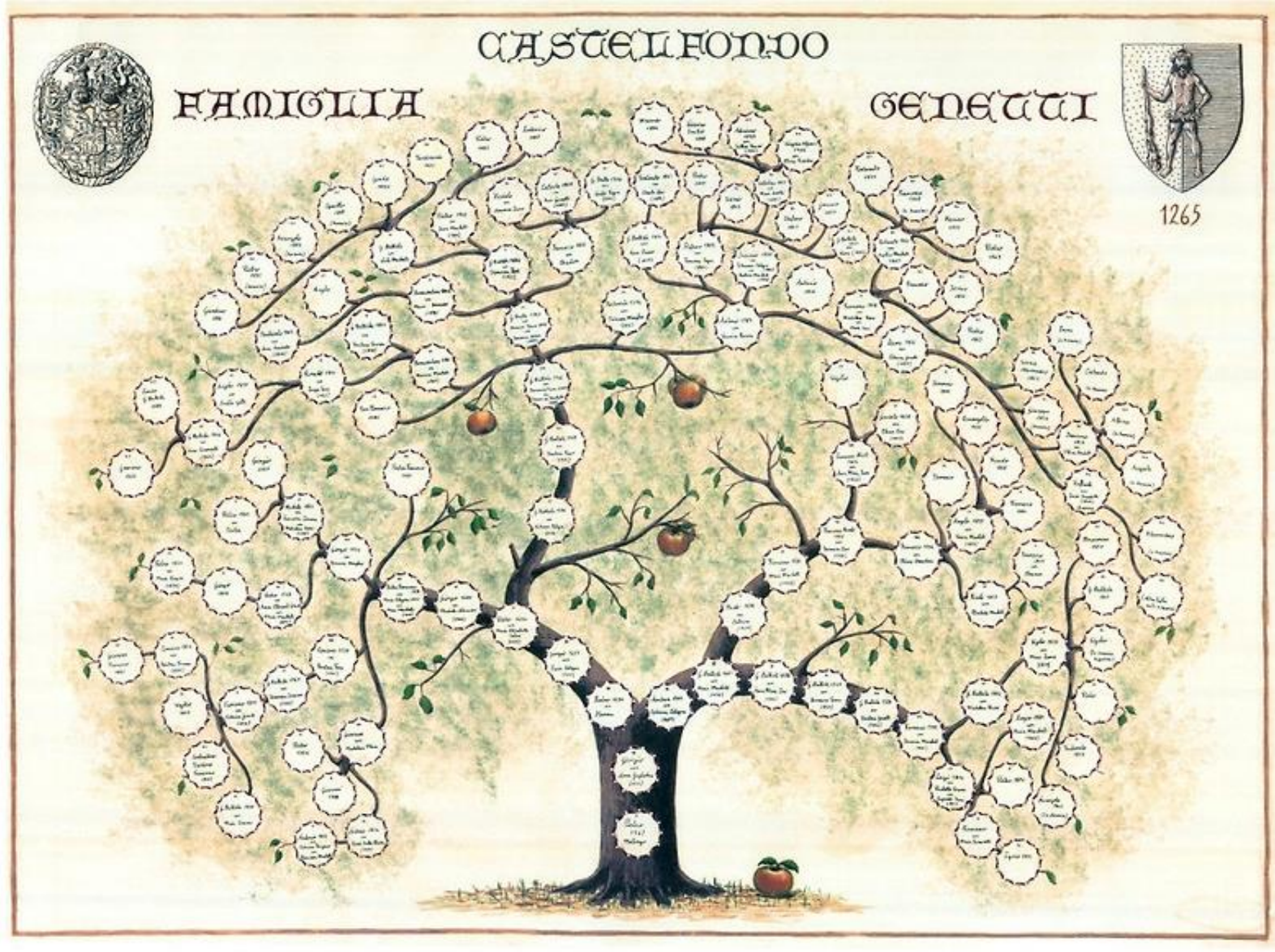
 copyright ©2024 m.patrignani

Example: maximal outerplanar dual



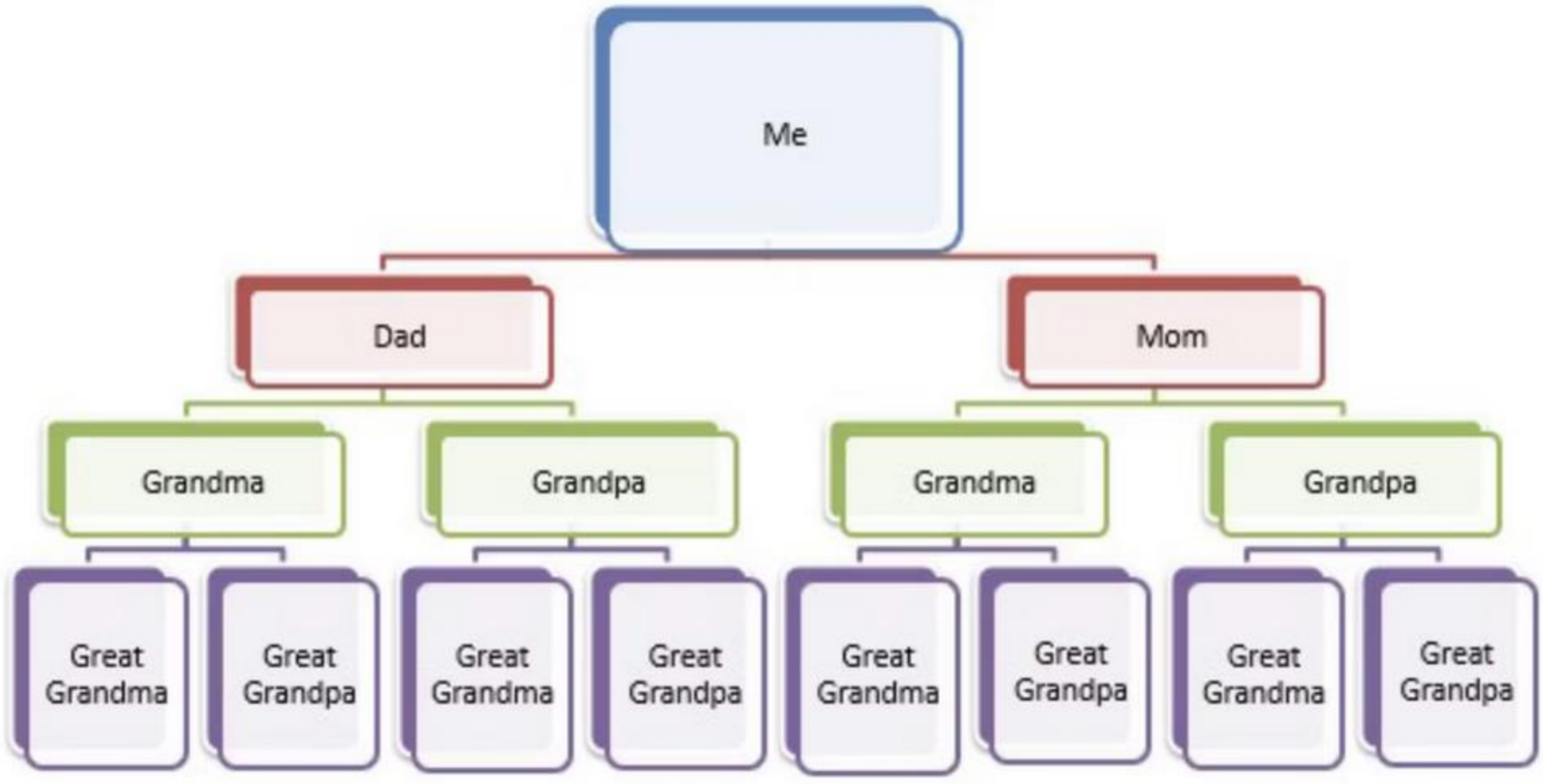
 copyright ©2024 m.patrignani

Example: genealogical tree



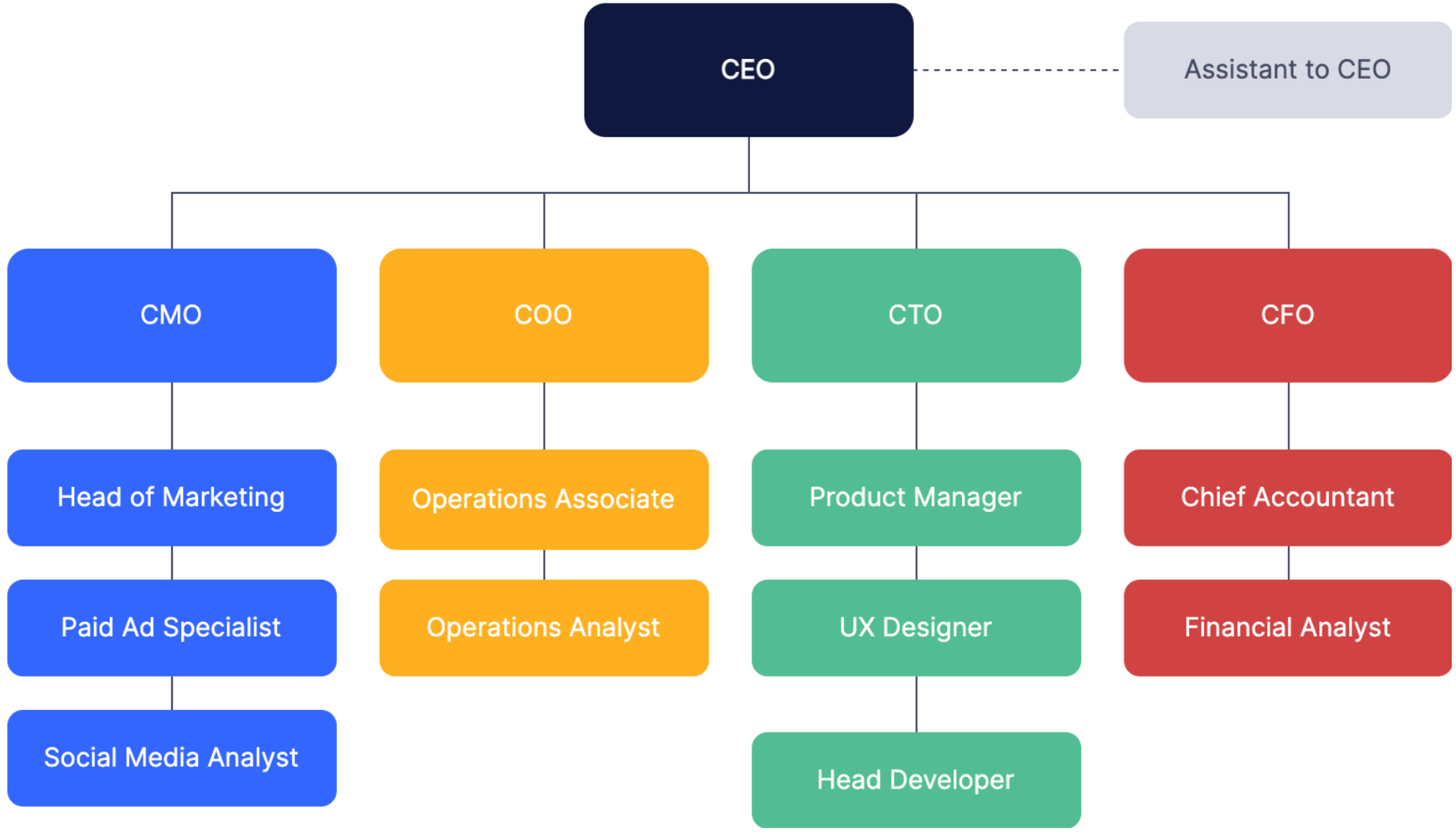
 copyright ©2024 m.patrignani

Example: family tree



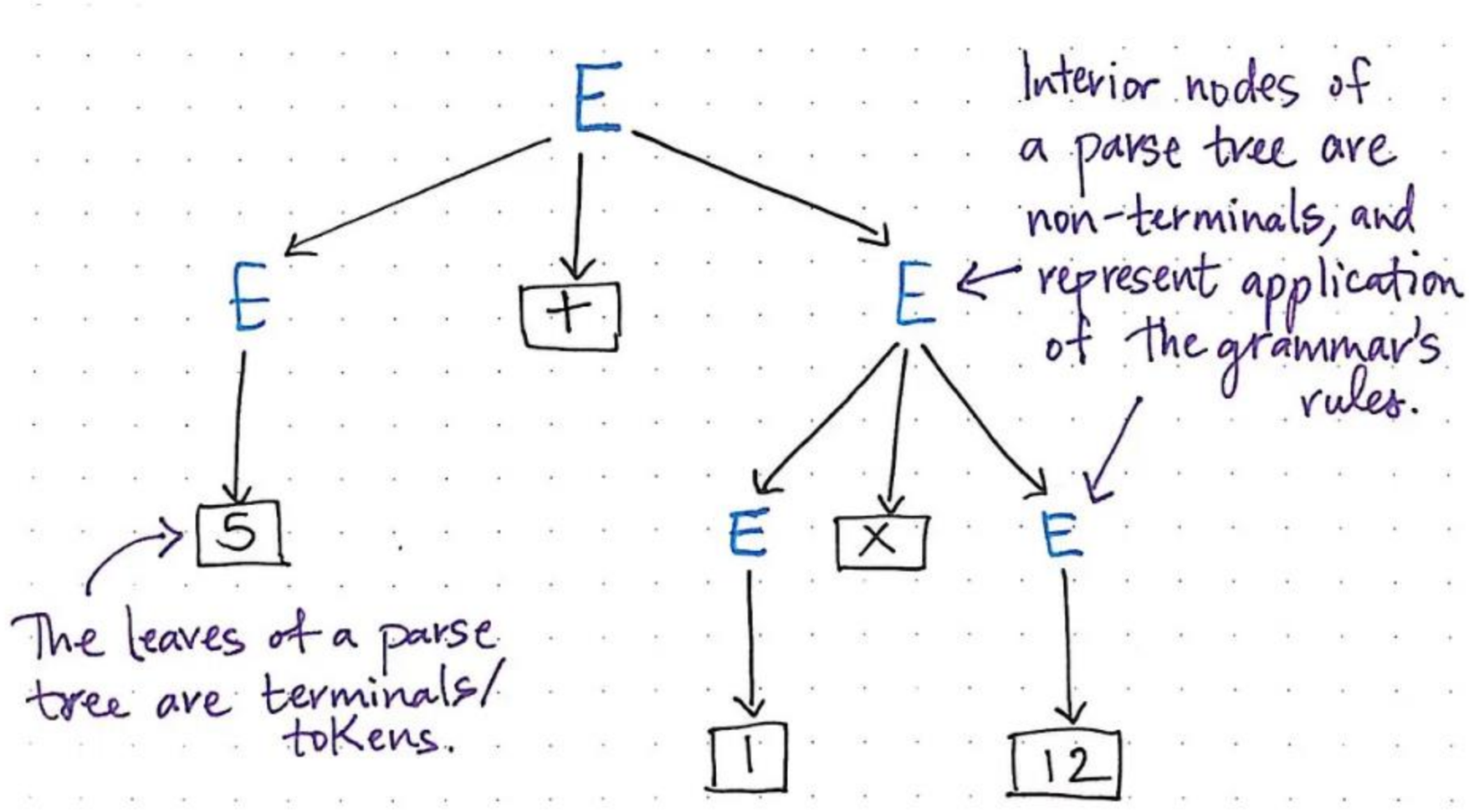
 copyright ©2024 m.patrignani

Example: organizational chart



 copyright ©2024 m.patrignani

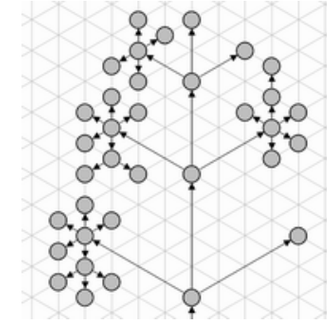
Example: parse trees

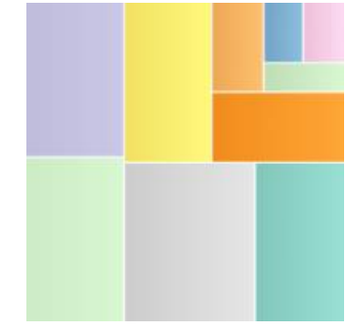


 copyright ©2024 m.patrignani

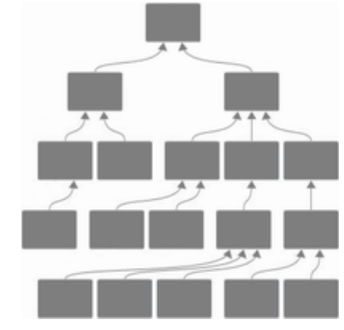
Tree representations

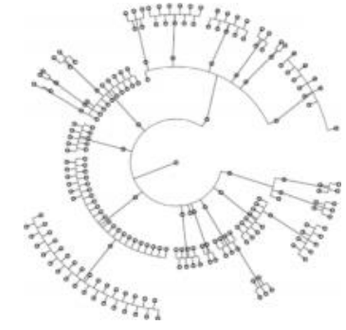
- Two main representation schemes
  - Node-link (this lecture)
  - Space-filling (next lecture)
- Node-link representations
  - Outline drawings
  - Layered drawings
  - Radial drawings
  - HV-drawings

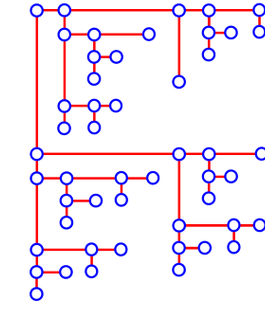








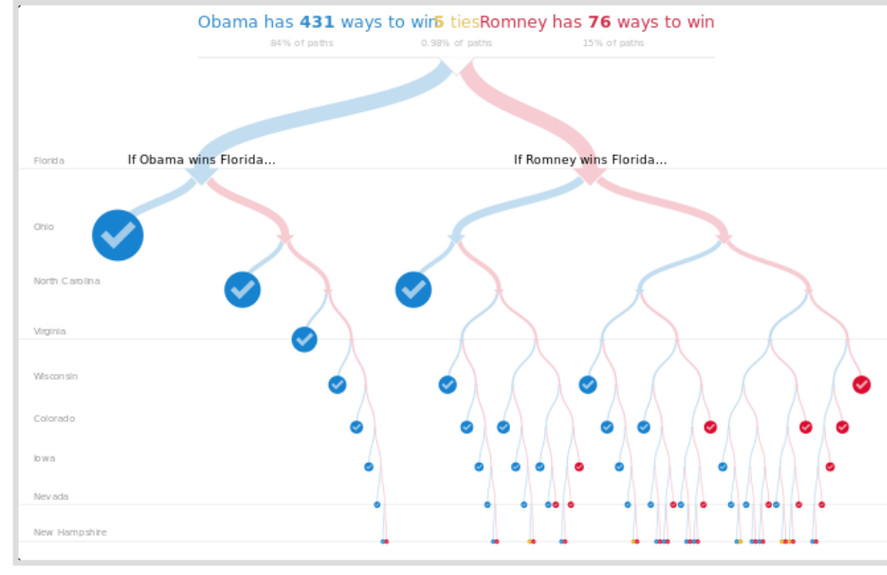




 copyright ©2024 m.patrignani

Node-link diagrams

- In node-link diagrams
  - the root is usually at the top
  - children are drawn below their parent

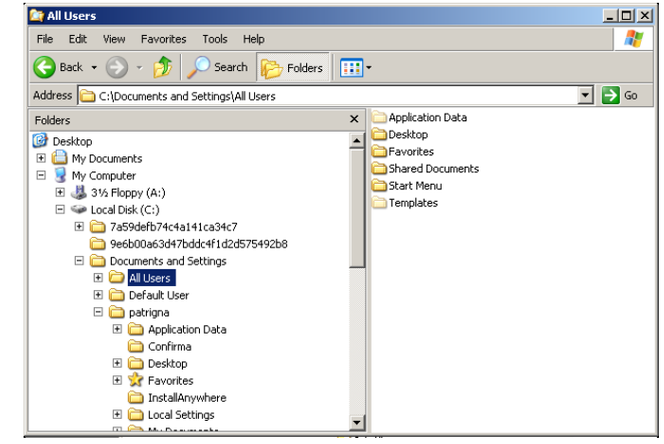


http://elections.nytimes.com/2012/results/president/scenarios

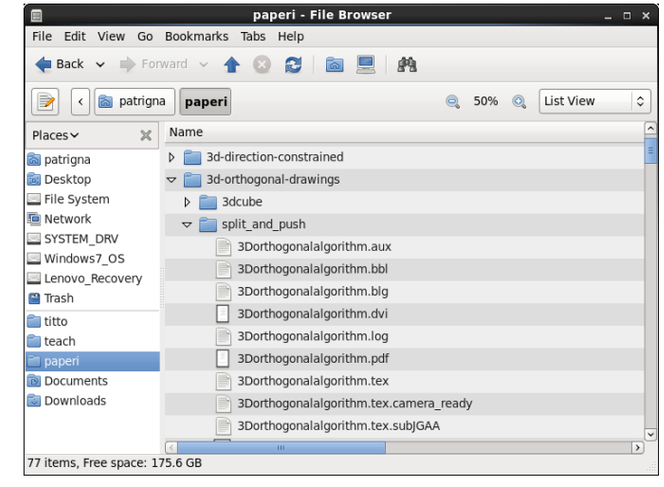
 copyright ©2024 m.patrignani

Outline drawings

- Tipical of file managers
  - allow navigation in the filesystem
- Pros and cons
  - good for drilling down in the tree
  - bad for searching
  - bad for conveying structure



Windows XP Explorer

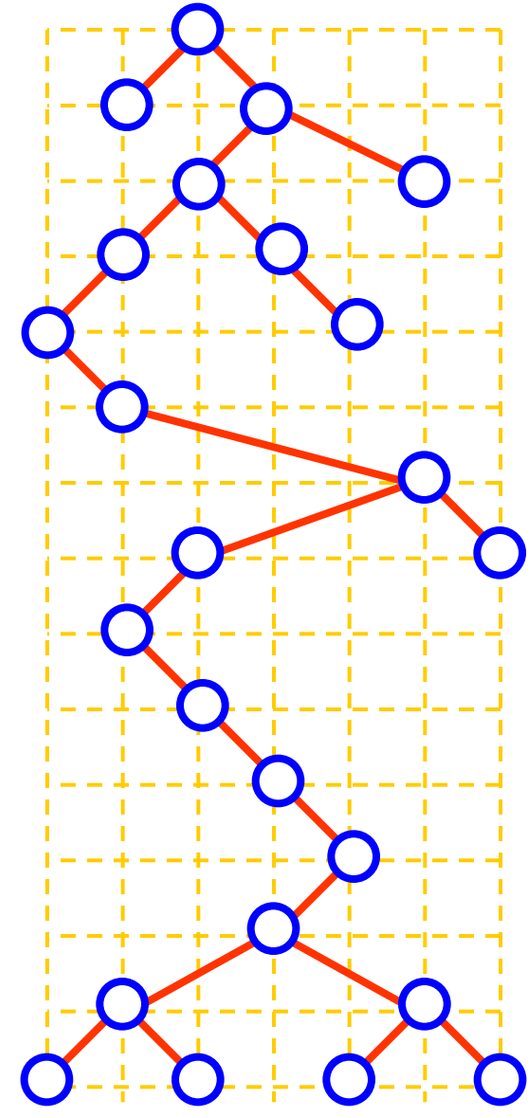


Linux Nautilus 2.28.4

copyright ©2024 m.patrignani

Layered drawings of binary trees

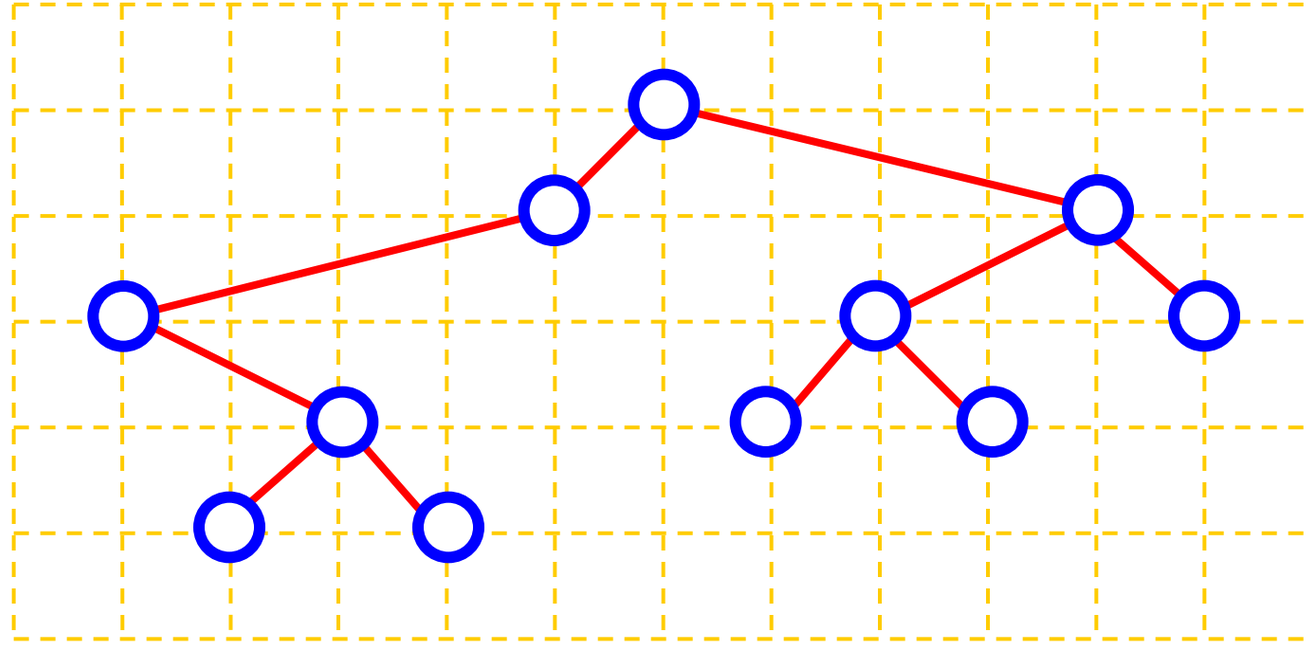
- Drawing convention (mandatory)
  - edges are non-crossing straight-line segments
  - y-coordinates are determined by nodes depth, with the root above the other nodes
  - the horizontal distance between two nodes is at least two



copyright ©2024 m.patrignani

What algorithm drew this tree?

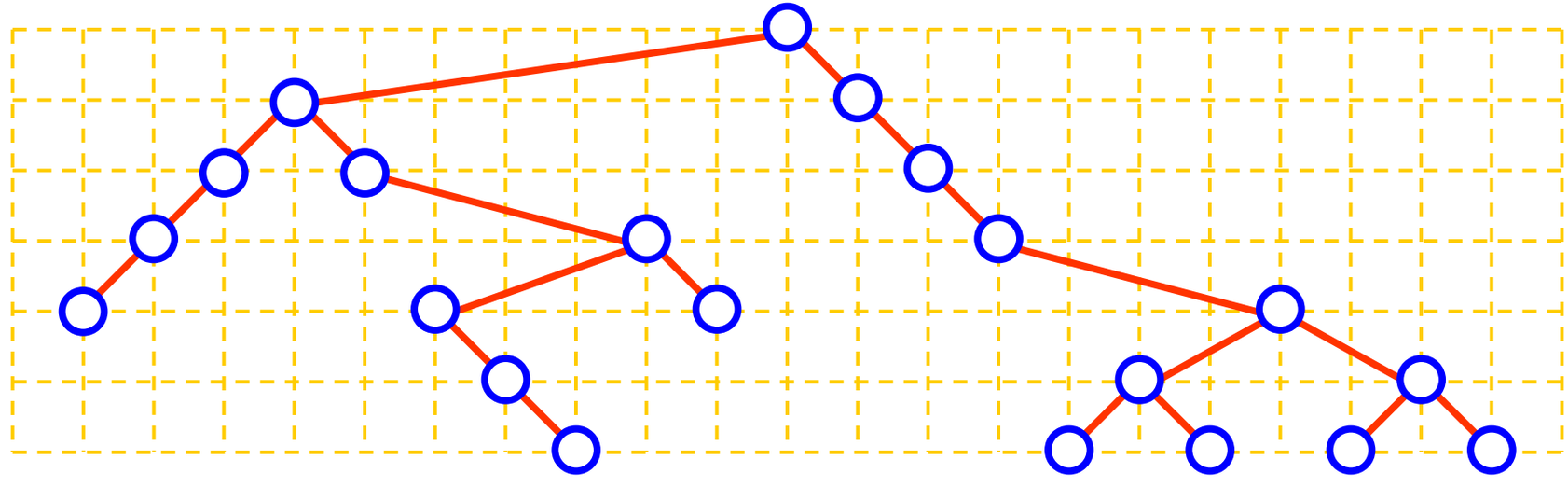
- How difficult is to compute y-coordinates?
- Can you guess how x-coordinates were computed?



 copyright ©2024 m.patrignani

Is this drawing satisfactory?

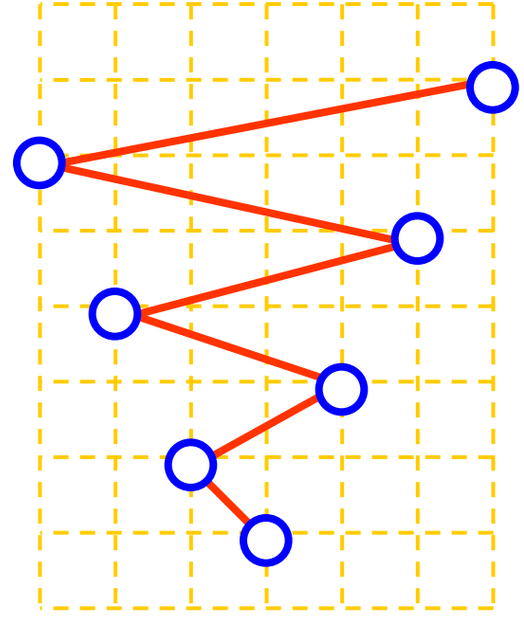
- The algorithm based on inorder traversal is due to [Knuth 71]
- At least two flaws
  - the drawing is usually much wider than necessary
  - a parent node is not necessarily centered with respect to its children



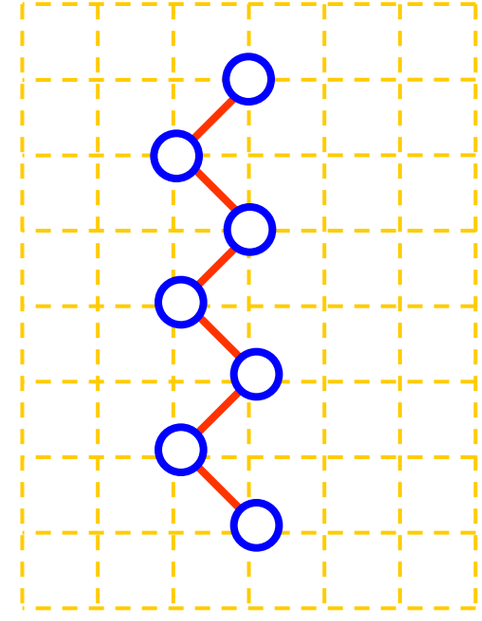
 copyright ©2024 m.patrignani

Inorder algorithm: a very bad case

- A path drawn with the inorder algorithm



Inorder algorithm

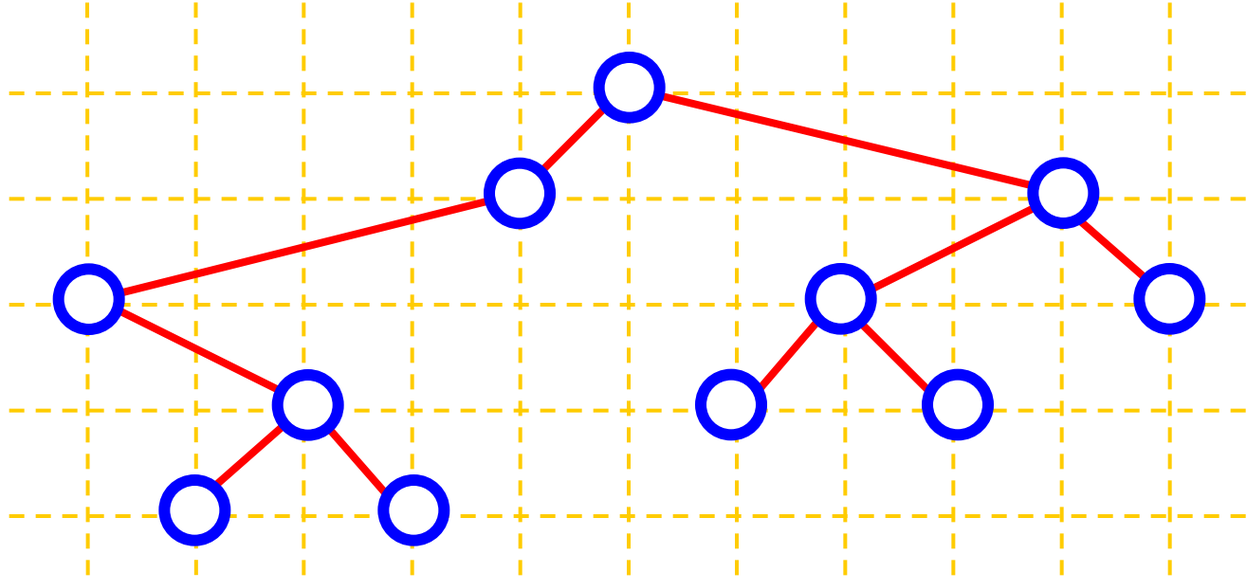


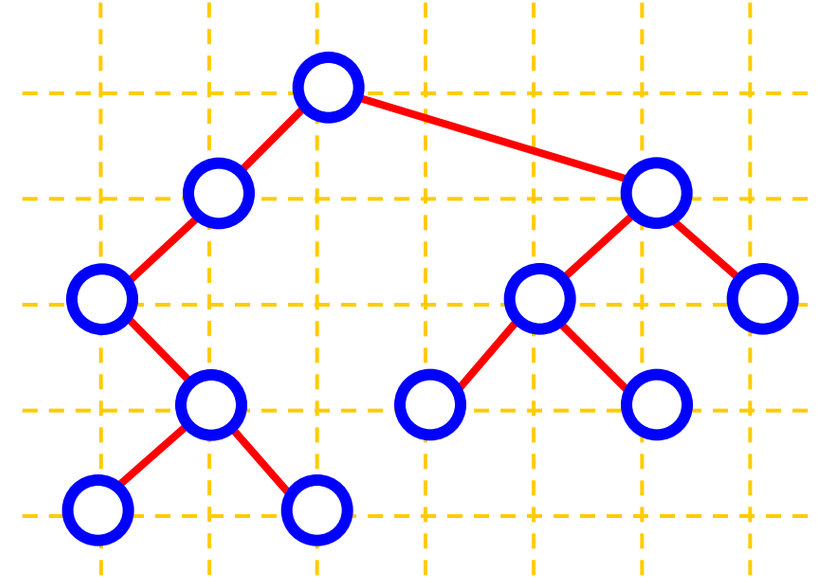
Optimal drawing

copyright ©2024 m.patrignani

First drawing aesthetic

1. Trees should be drawn as narrowly as possible
   - reduced width implies reduced area

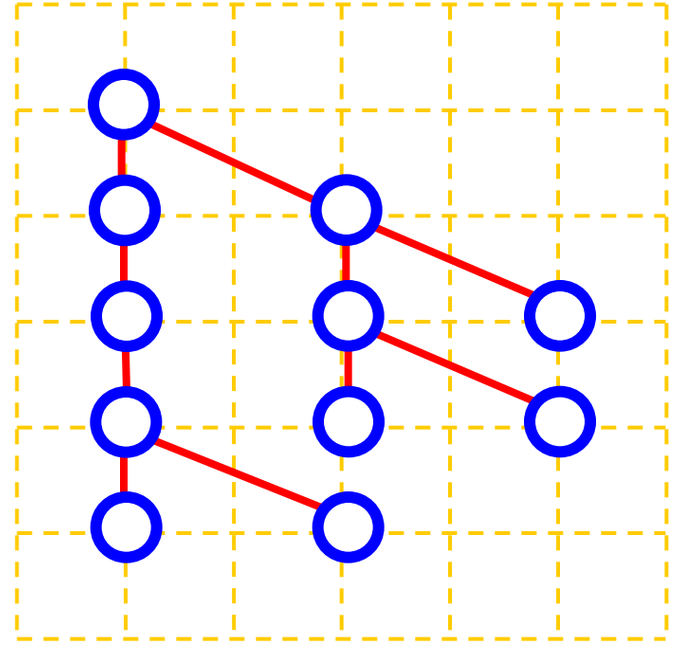




 copyright ©2024 m.patrignani

A new simple algorithm

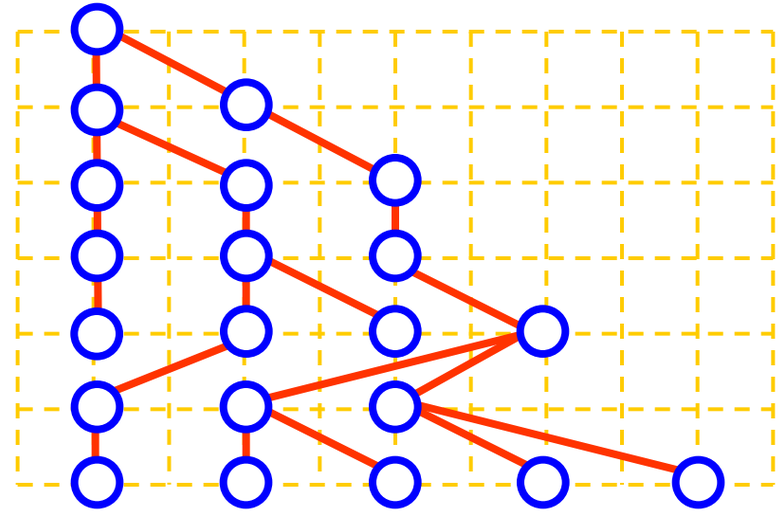
- Introduced by [Wetherell & Shannon 79]
- Can you guess how x-coordinates were computed?



 copyright ©2024 m.patrignani

Is this drawing satisfactory?

- The algorithm based on postorder traversal by [Wetherell & Shannon 79] as at least two flows
  - the drawing is usually ugly
  - a parent node is not necessarily centered with respect to its children
    - sometimes a left child or a right child are not recognizable



 copyright ©2024 m.patrignani

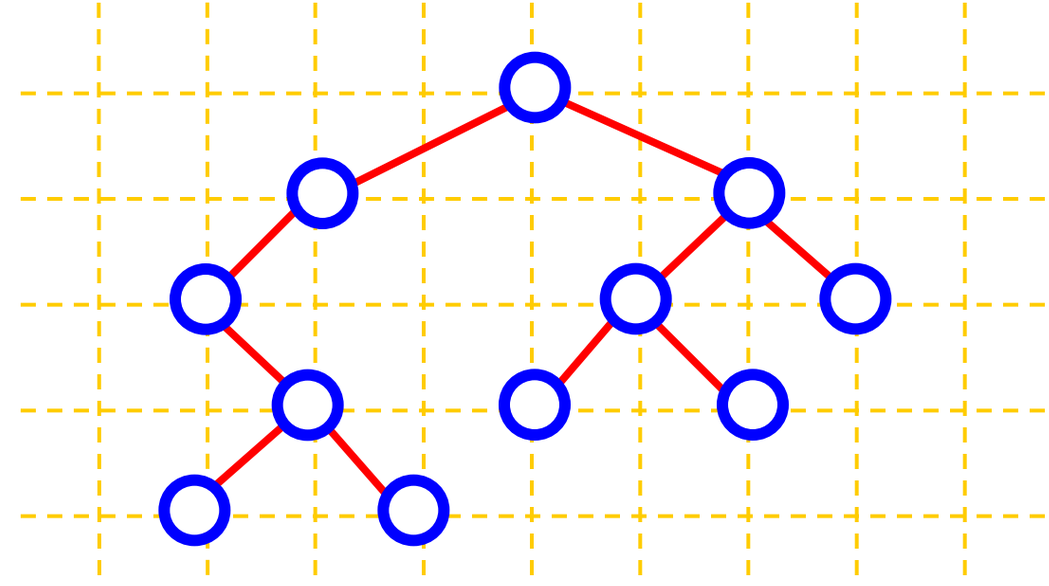
Reformulation of our problem

Drawing conventions (mandatory)

- edges are non-crossing straight-line segments
- y-coordinates are determined by nodes depth, with the root above the other nodes
- the horizontal distance between two nodes is at least two
- each parent is centered between its children
  - its x-coordinate is the average of theirs coordinates
- isomorphic subtrees are displayed isomorphic

Drawing aesthetics

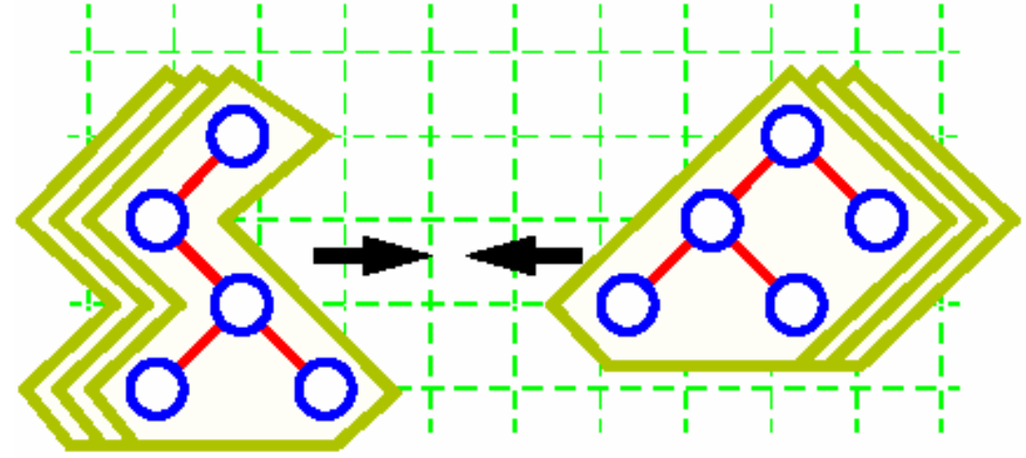
- Trees should be drawn as narrowly as possible

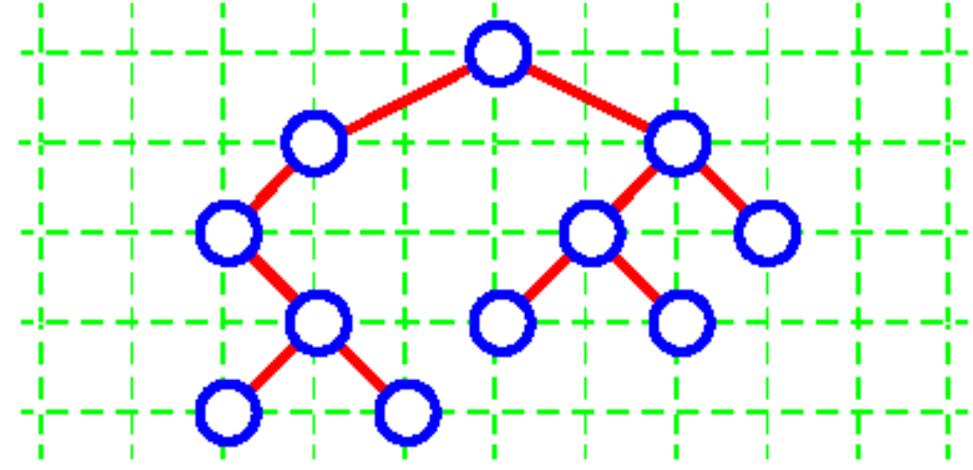


copyright ©2024 m.patrignani

Idea for an (efficient) algorithm

- Draw each subtree separately and then attach the two drawings
  - [Reingold &Tilford 81]





copyright ©2024 m.patrignani

More concretely

- We need a bottom-up traversal of the tree
- We should be able to “translate” the subtrees
  - the relative position of the subtrees can be described by the horizontal distance of their roots with respect to their parents
- Suppose all relative positions of children with respect to their parents are known
  - how do you compute the final x-coordinates of the nodes?

 copyright ©2024 m.patrignani

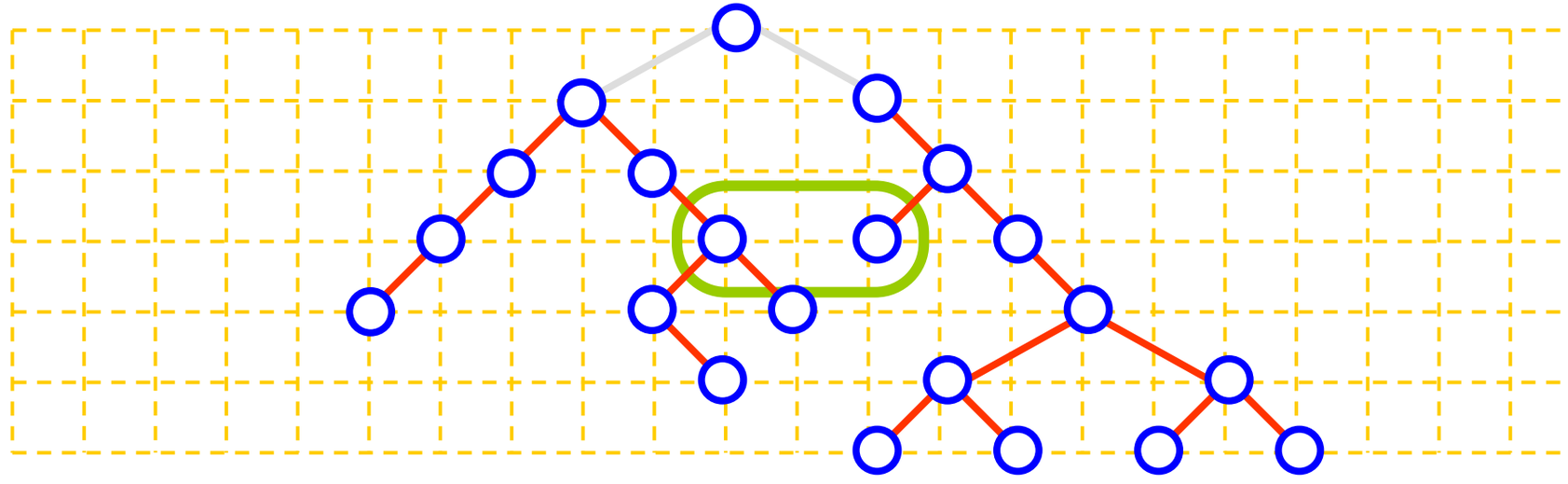
Bottom-up (postorder) traversal

- Base cases
  - if *v* is a leaf nothing to do
- Divide
  - if *v* is not a leaf recursively launch the traversal on the left and right subtrees
- Conquer
  - if *v* has only one child, say the left one, then place the child at horizontal distance 1 to the left of *v*
  - if *v* has both children? Complex case!

 copyright ©2024 m.patrignani

Complex case

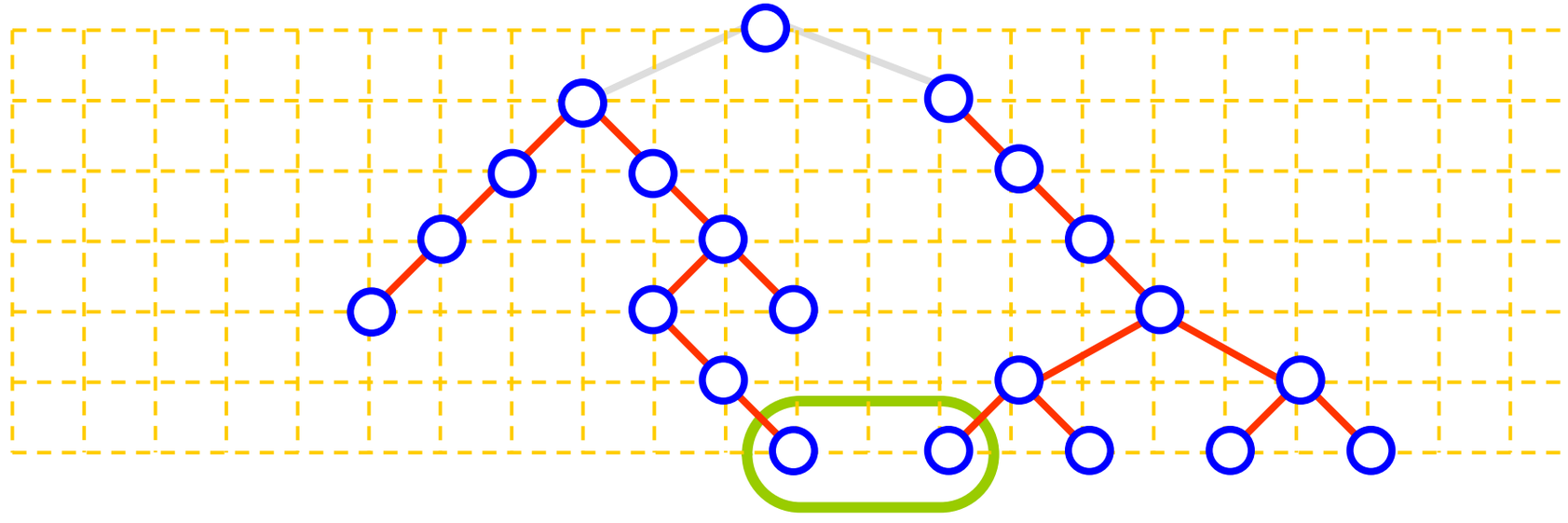
- If *v* has both children
  - move the drawings of the two subtrees towards each other until their minimum horizontal distance is two
  - place *v* in the middle



 copyright ©2024 m.patrignani

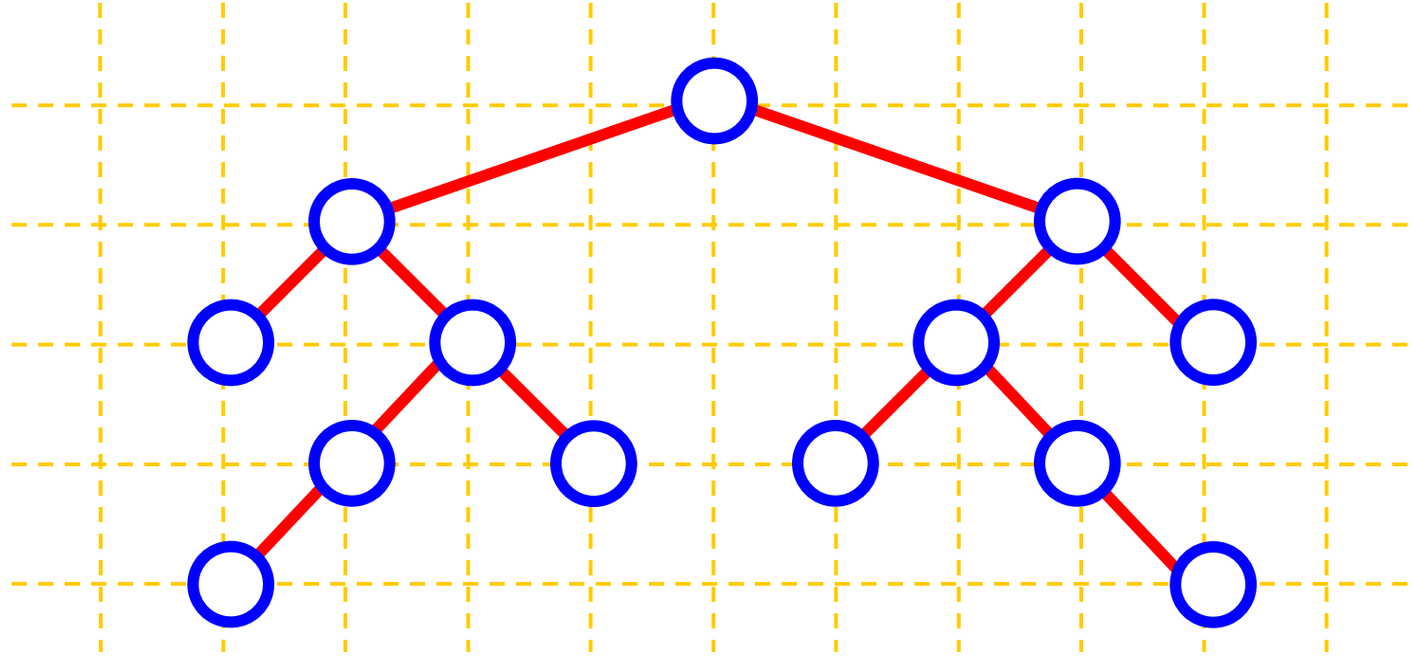
Complex case

- What if the children have odd distance
  - putting the parent in the middle would produce non-integer x-coordinates for *v*!
  - what then?
    - either you allow that or you move the subtrees one further unit apart



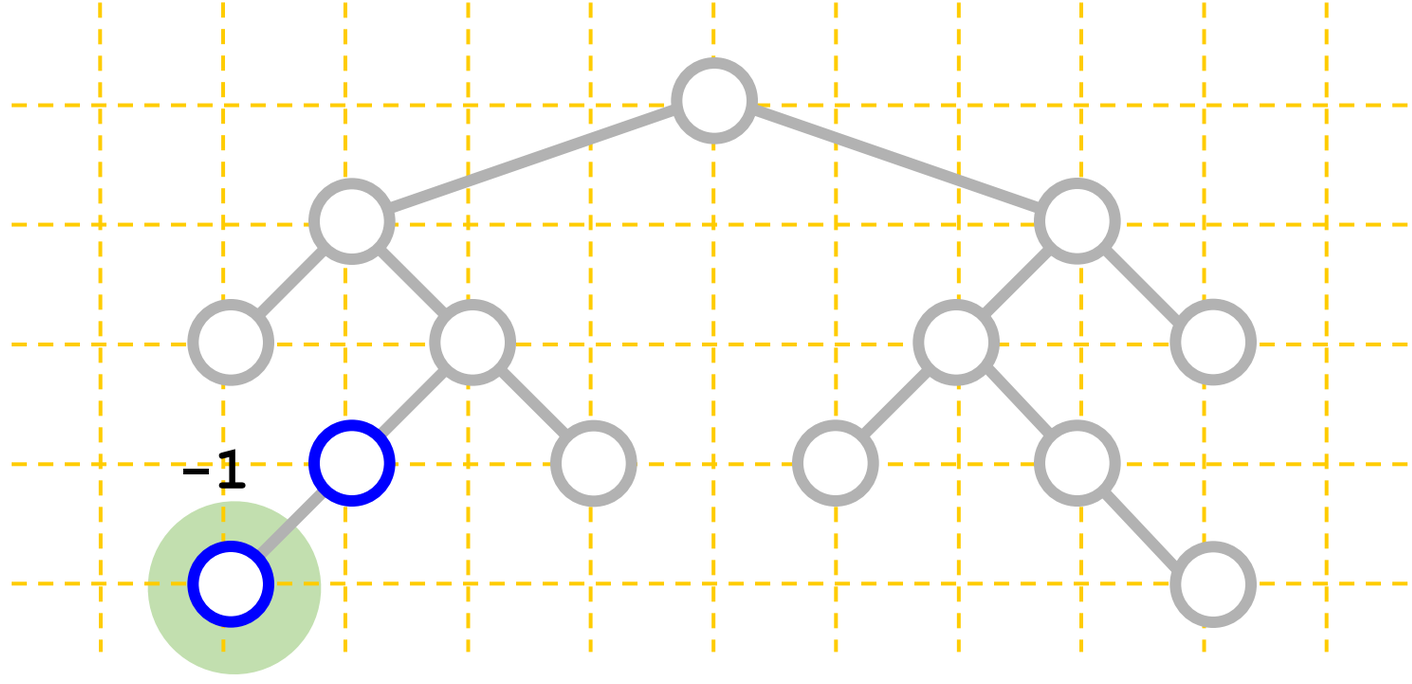
 copyright ©2024 m.patrignani

Example of postorder traversal



copyright ©2024 m.patrignani

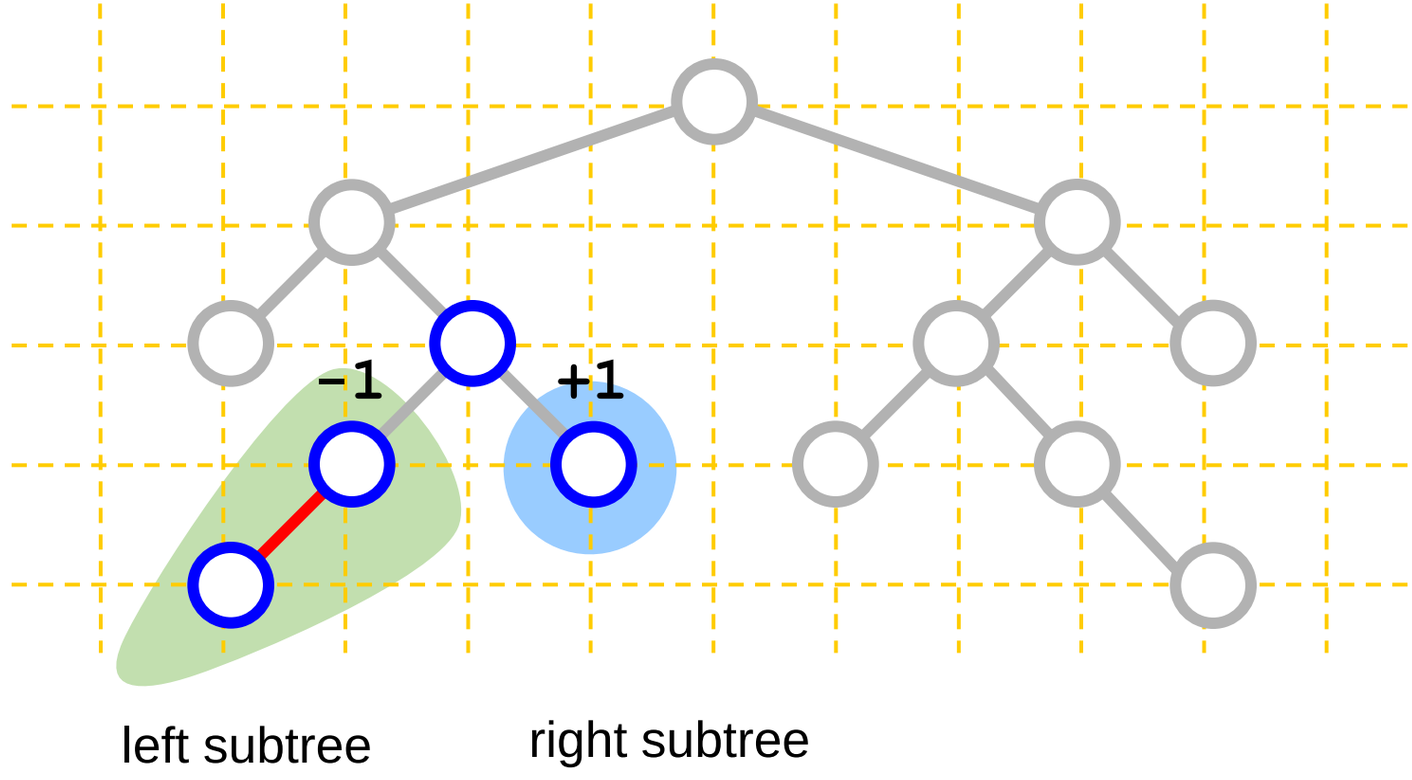
Example of postorder traversal



left subtree

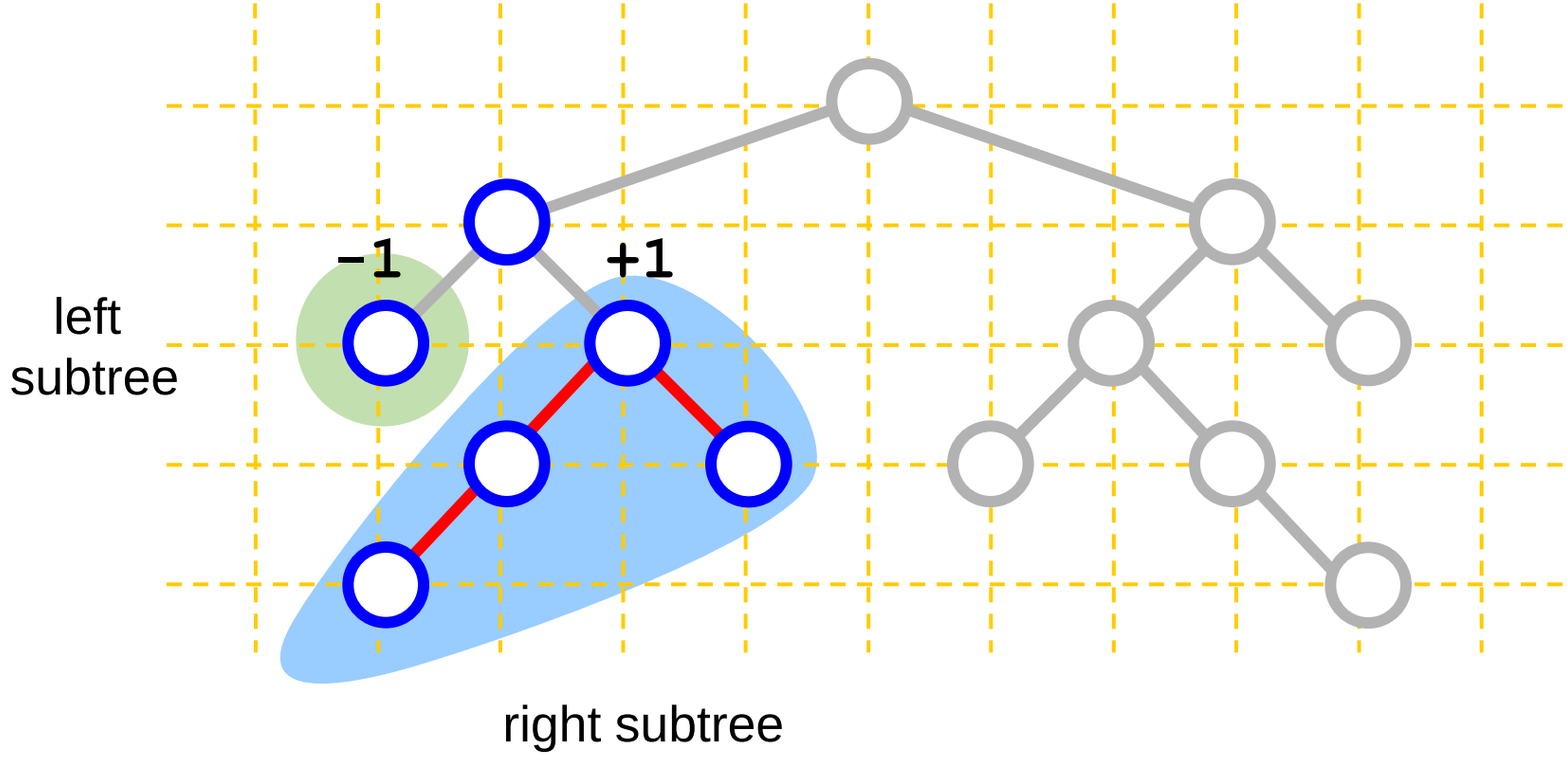
 copyright ©2024 m.patrignani

Example of postorder traversal



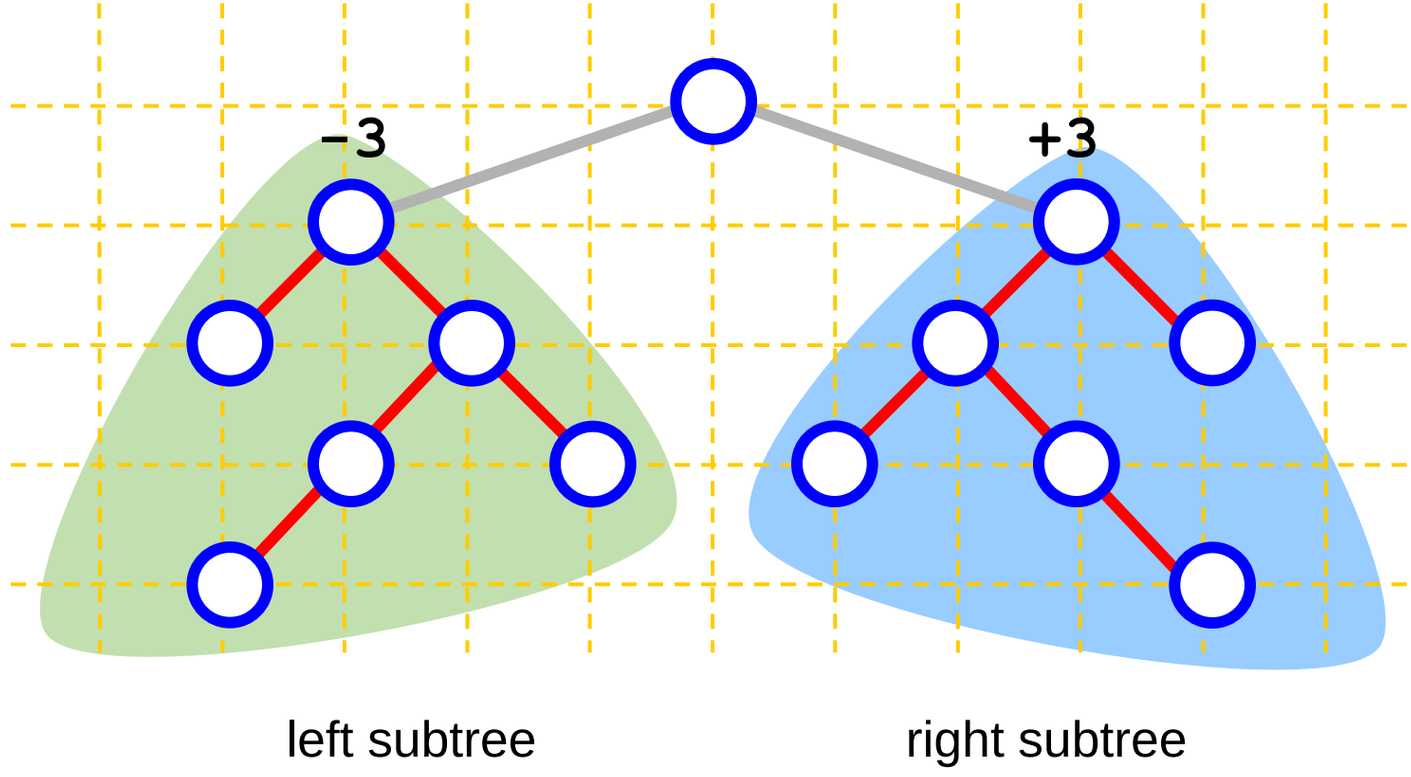
 copyright ©2024 m.patrignani

Example of postorder traversal



copyright ©2024 m.patrignani

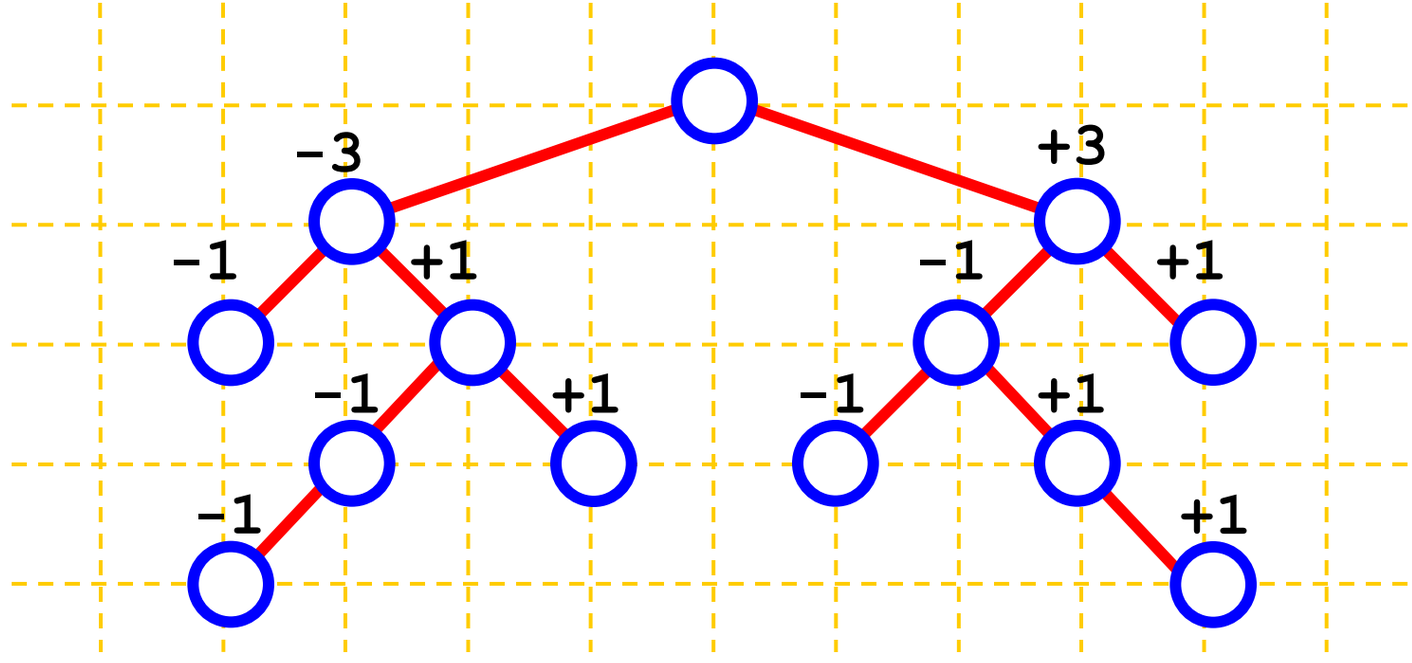
Example of postorder traversal



 copyright ©2024 m.patrignani

Example of postorder traversal

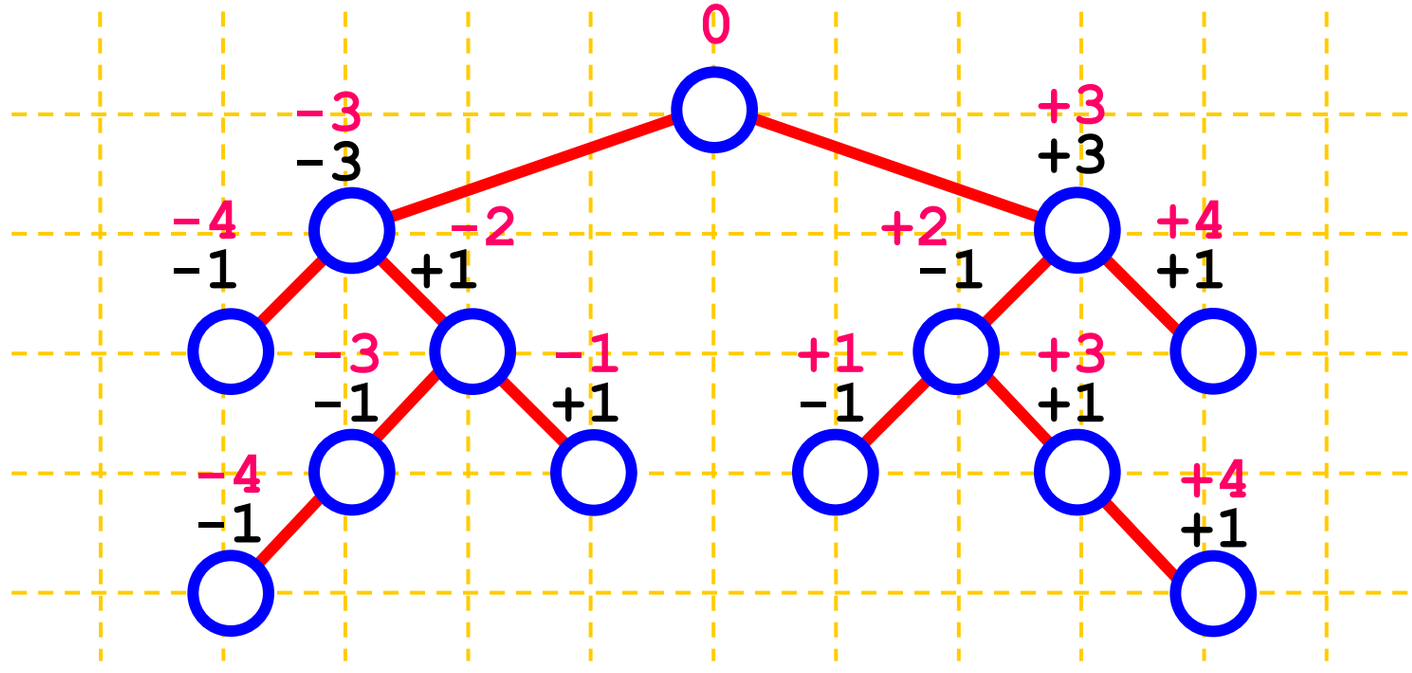
after the postorder traversal



copyright ©2024 m.patrignani

Example of postorder traversal

after the postorder traversal

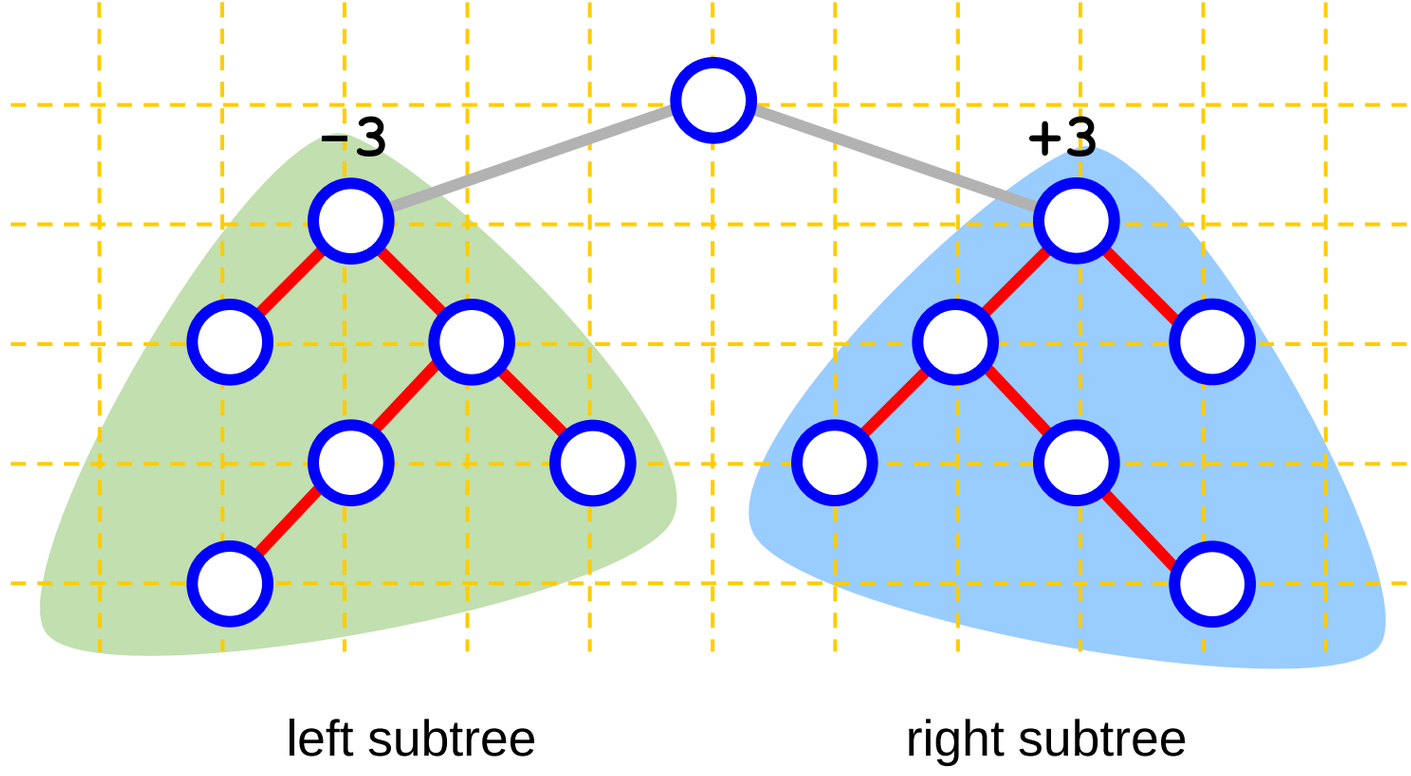


after the preorder traversal

 copyright ©2024 m.patrignani

Towards determining complexity

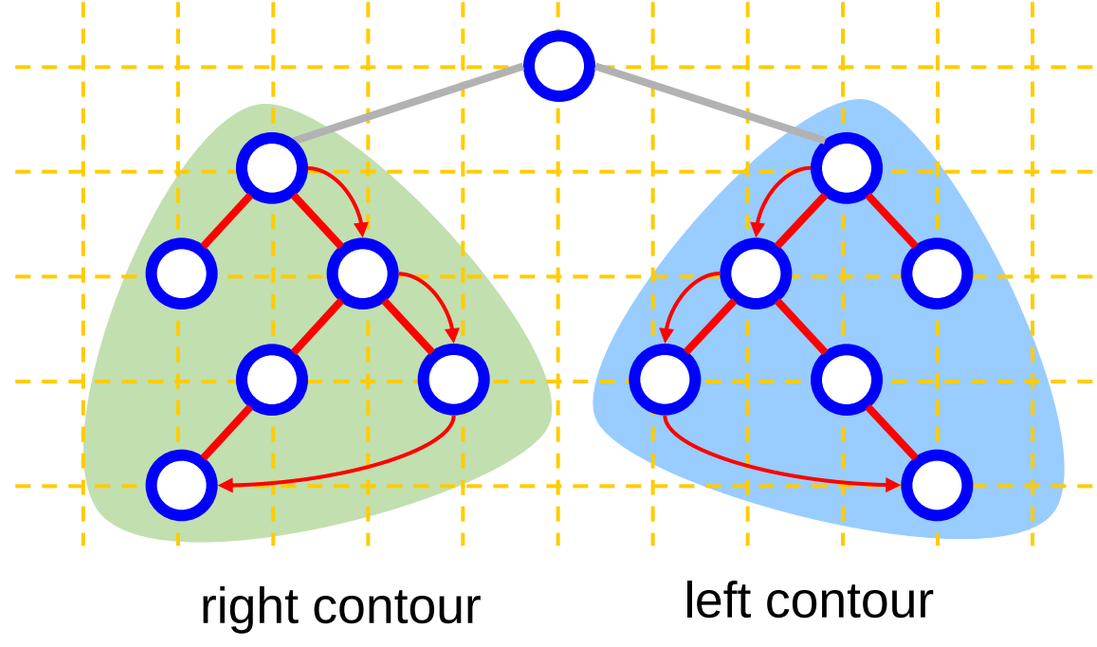
- What is the heaviest step of the computation?
- How heavy is it?



 copyright ©2024 m.patrignani

Left and right contours

- The *left contour* of a binary tree is the sequence of vertices that are the leftmost of those with the same level
- The *right contour* is similarly defined
- Left and right contours can be stored in linked lists



 copyright ©2024 m.patrignani

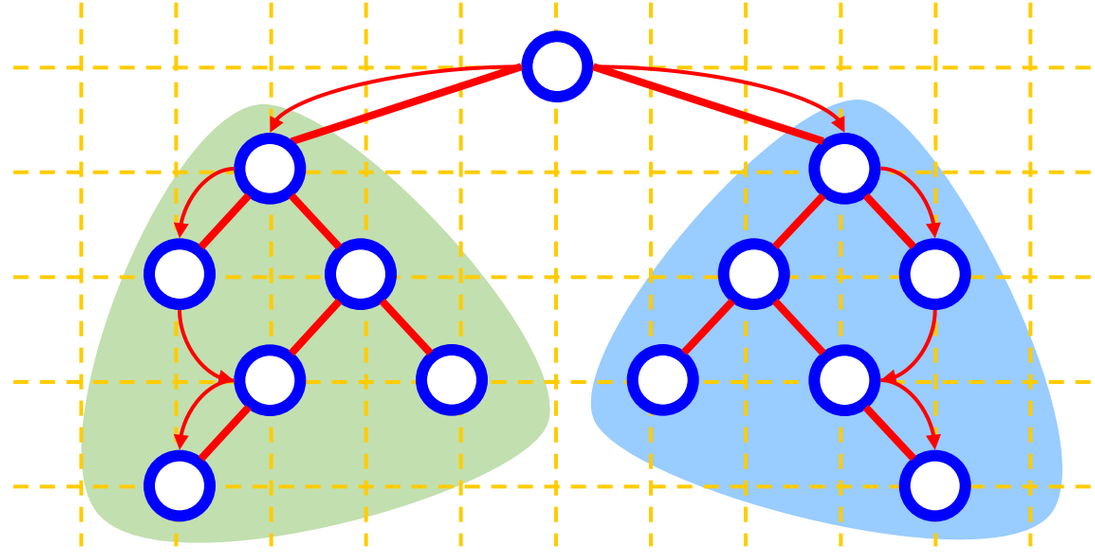
Comparing left and right contours

- Processing node *v* in postorder traversal
  - scan the right contour of the left subtree of *v* and the left contour of the right subtree of *v*
  - accumulate the displacements of the vertices encountered on the left and right contours and keep track of the maximum cumulative displacement at any depth
  - obtain the distance at which the roots of the subtrees have to be put in order to have minimum distance 2

 copyright ©2024 m.patrignani

Updating left and right contours

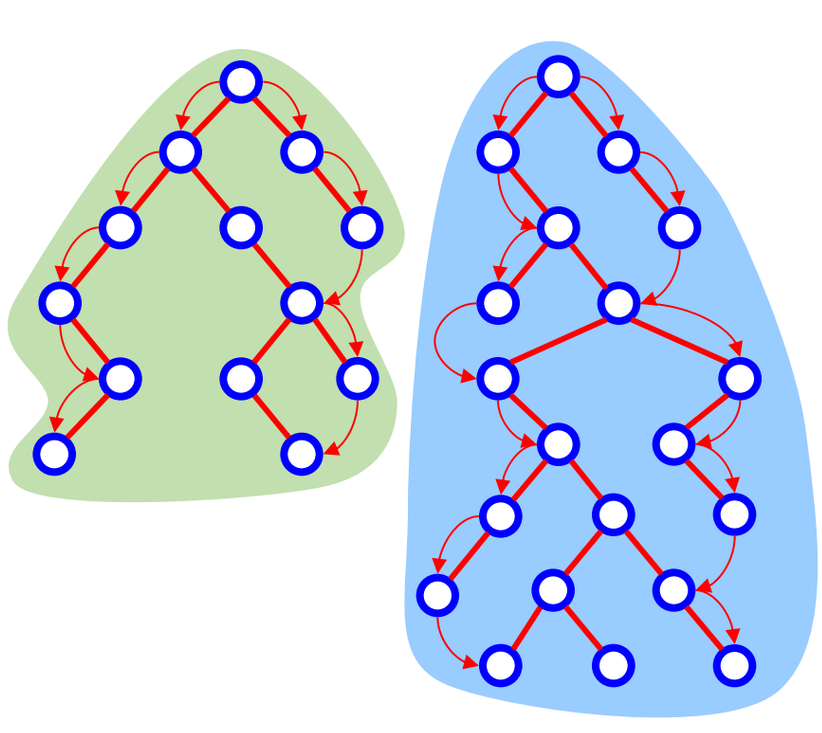
- If the two subtrees of *v* have the same height, then
  - the left contour of *v* is the left contour of its left child plus vertex *v* itself
  - the right contour of *v* is the right contour list its right child plus *v* itself

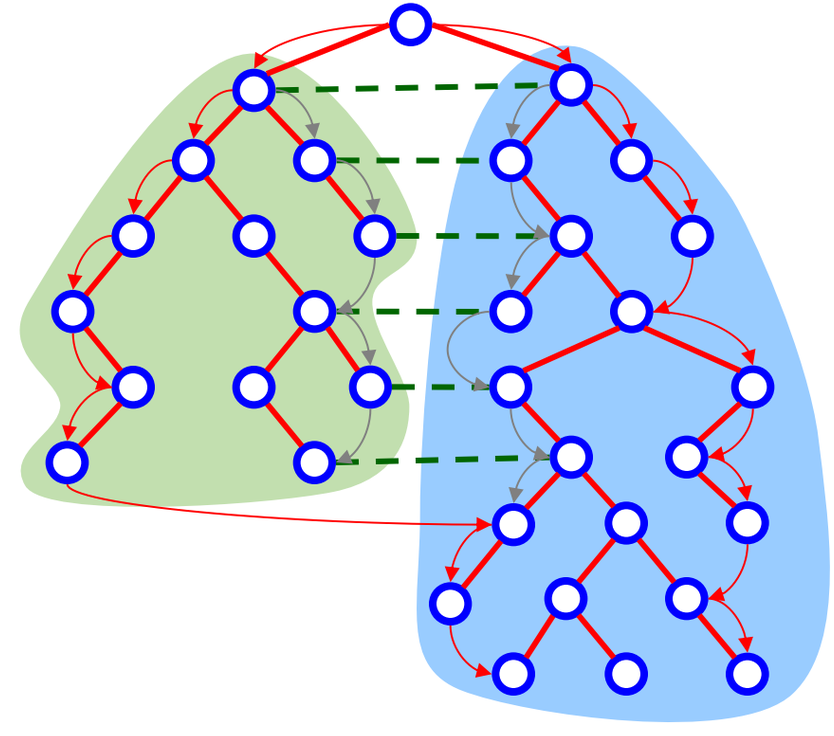


 copyright ©2024 m.patrignani

Updating left and right contours

- If the height of the two subtrees is different
  - the left and right contours can be obtained by suitably glueing pieces of the contours of the two subtrees





 copyright ©2024 m.patrignani

A crucial observation for efficiency

- It is necessary to travel down the contours of the subtrees only as far as the height of the subtree of lesser height
  - the time spent for processing vertex *v* in the postorder traversal is proportional to the minimum of the heights of the subtrees of *v*

$$1 + \min\{h'(v), h''(v)\}$$

  - the running time of the postorder traversal is

$$\Sigma_{v \in T} (1 + \min\{h'(v), h''(v)\})$$

$$n + \Sigma_{v \in T} \min\{h'(v), h''(v)\}$$

 copyright ©2024 m.patrignani

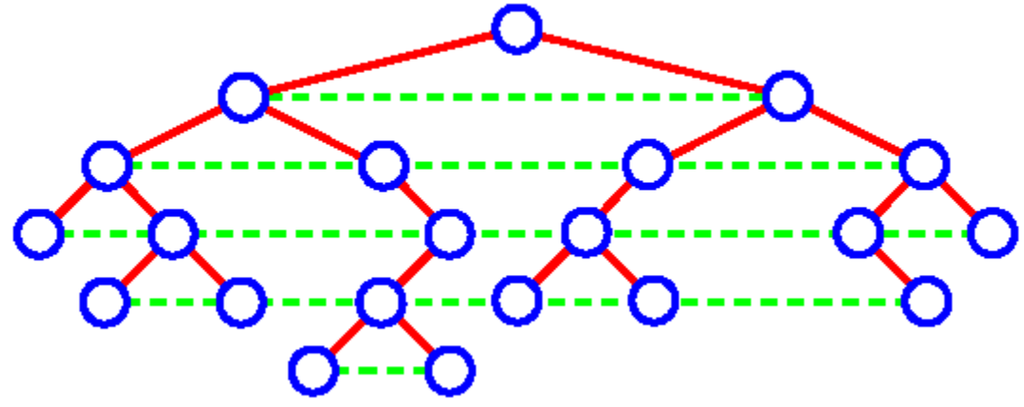
A “pictorial” proof of complexity

- We can visualize the sum

$$\Sigma_{v \in T} \min\{h'(v), h''(v)\}$$

by connecting with new edges pairs of consecutive vertices with the same depth

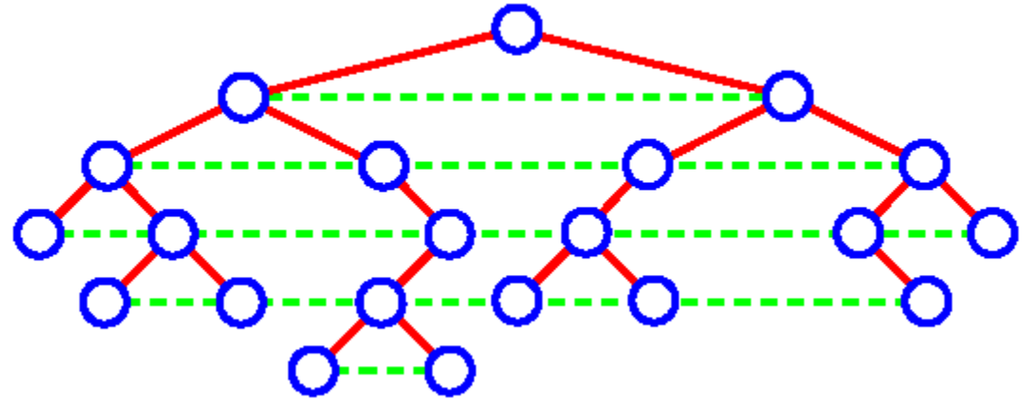
  - the sum over all vertices v of the minimum height of the subtrees of v is equal to the number of green edges added to the tree



 copyright ©2024 m.patrignani

A “pictorial” proof of complexity

- Each vertex is incident to at most one green edge on its right
- The number of green edges (and therefore sum $\Sigma_{v \in T}$ min{h’(v),h”(v)}) is no more than the number of vertices of the tree
- The total complexity is linear



 copyright ©2024 m.patrignani

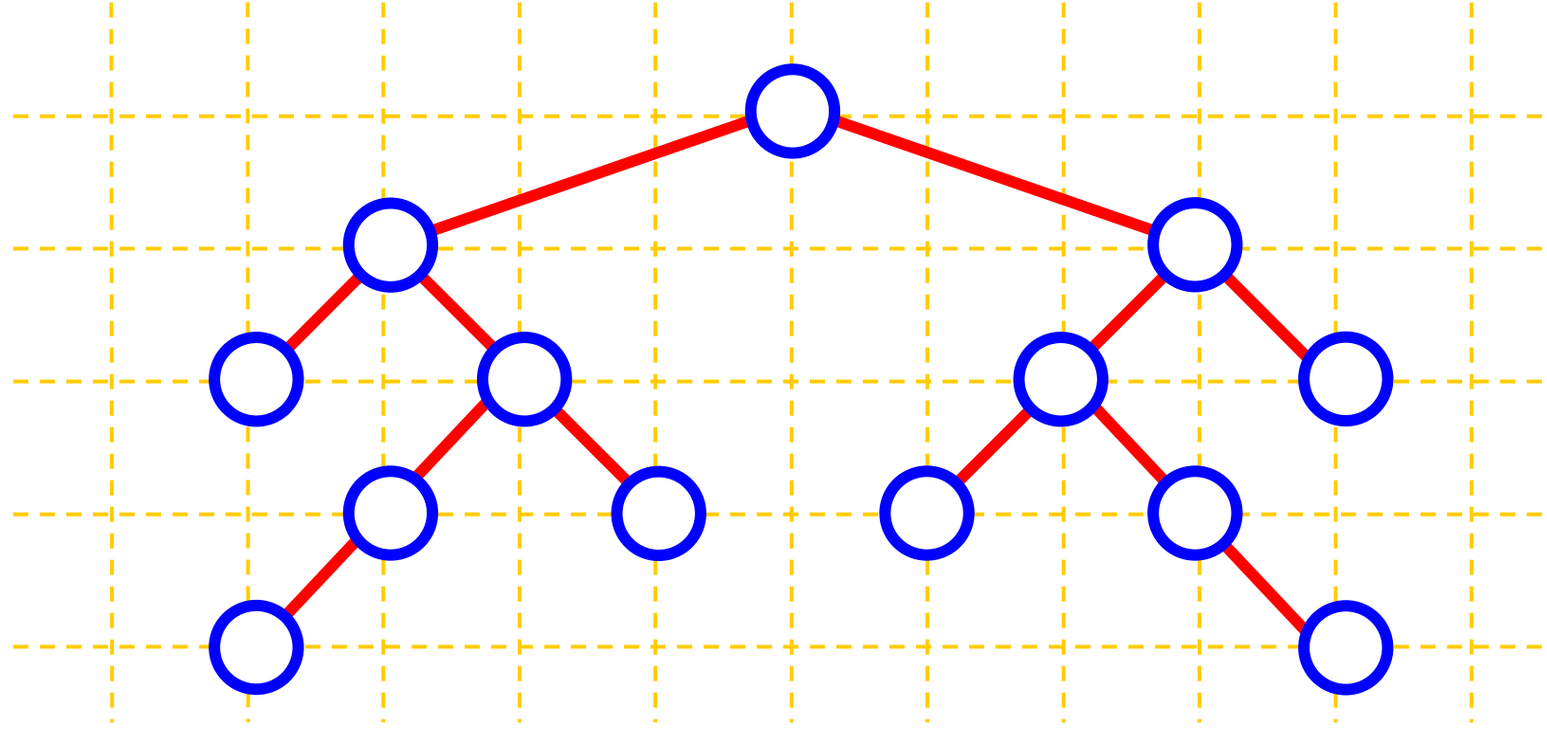
Properties of the algorithm

- Performances
  - linear-time algorithm ✓
- Conventions
  - edges are straight-lines ✓
  - no crossings among edges ✓
  - nodes have integer coordinates ✓
  - y-coordinates are determined by nodes depth ✓
  - left-right ordering of children respected ✓
  - parent centralized above its children ✓
- Aesthetics
  - isomorphic subtrees are displayed isomorphic ✓
  - minimum width (i.e., minimum area) ×

 copyright ©2024 m.patrignani

What is the drawing area?

- Height is fixed
  - is it O(n)?
  - Why?
- Width
  - is it O(n)?
  - Why?



copyright ©2024 m.patrignani

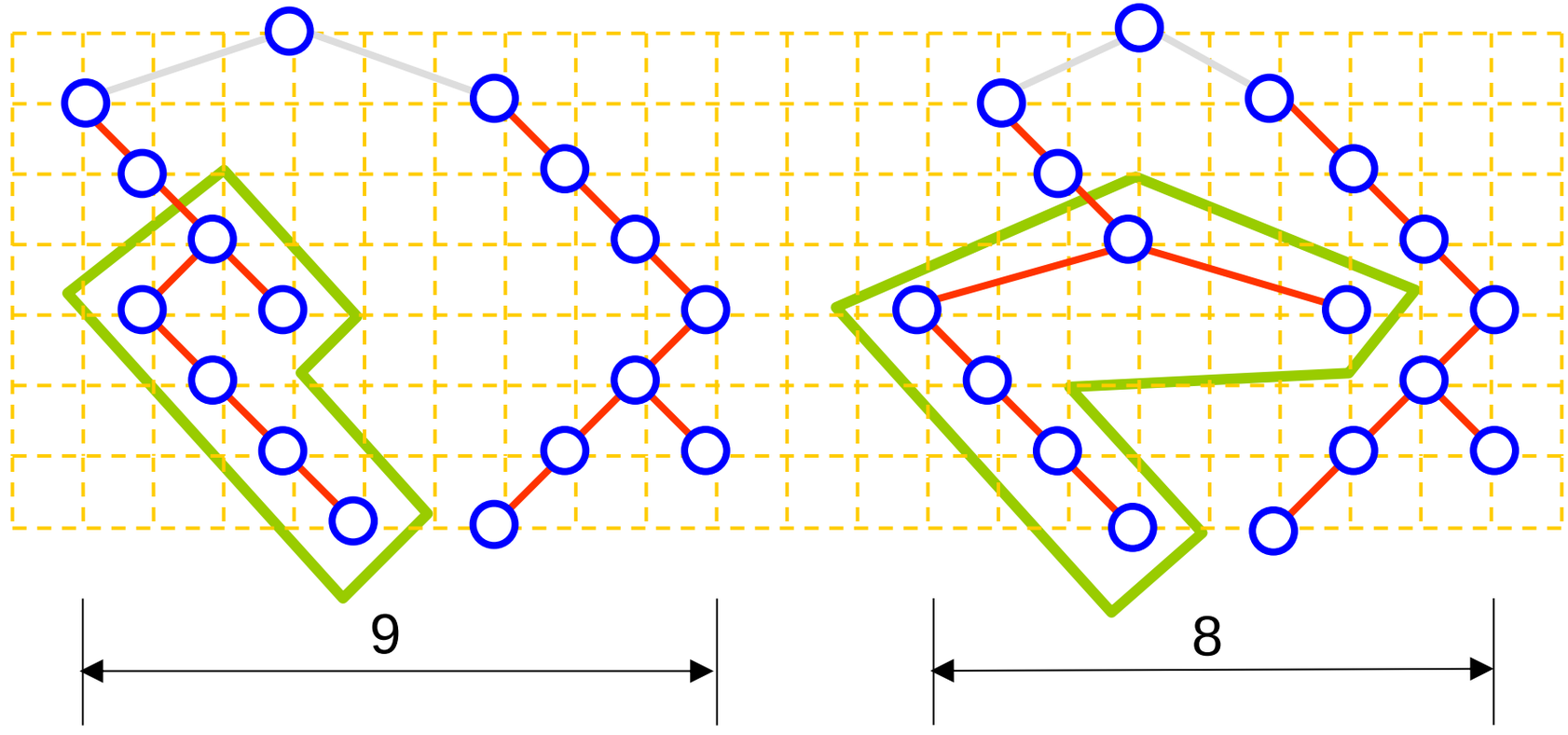
Is the area asymptotically optimal?

- Does there exist an input which always has height $\Omega(n)$ and width $\Omega(n)$ whatever is the algorithm we use?
  - if yes, the area is asymptotically optimal

 copyright ©2024 m.patrignani

Is the area the minimum possible?

- Local optimality is not global optimality!



 copyright ©2024 m.patrignani

Minimum area requirement

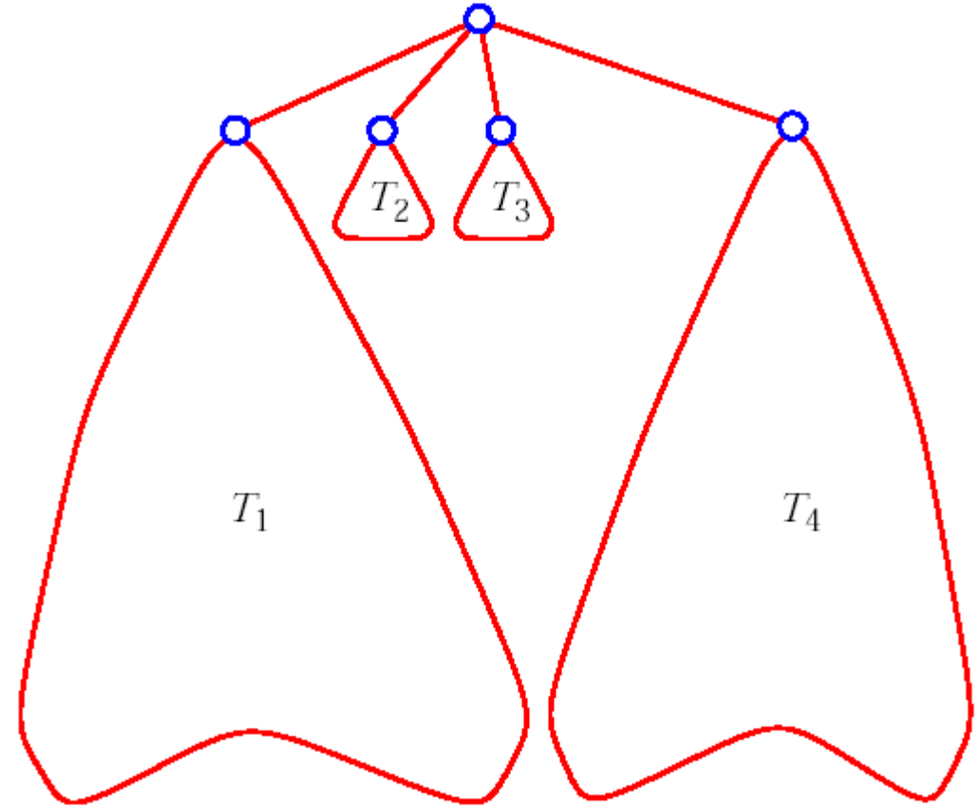
- Reingold and Tilford algorithm does not meet the requirement of minimum area
  - however the $O(n^2)$ area is asymptotically optimal
- There cannot be a polynomial-time algorithm to produce drawings of minimum area (unless P=NP)
  - proved in [Supowit & Reingold, 82]

 copyright ©2024 m.patrignani

Extensions

- Handling trees of arbitrary degree
  - [Walker 90], improved to linear by [Buchheim *et al.*, 06]

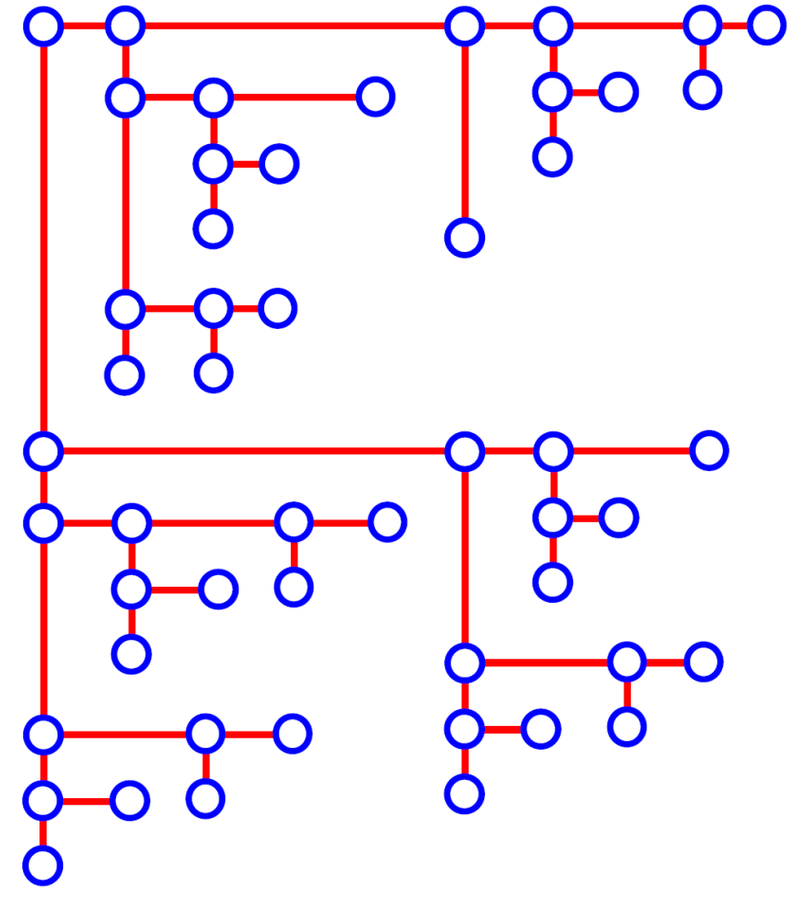
Figure from
[Di Battista *et al.* 99]



 copyright ©2024 m.patrignani

hv-drawings of unordered binary trees

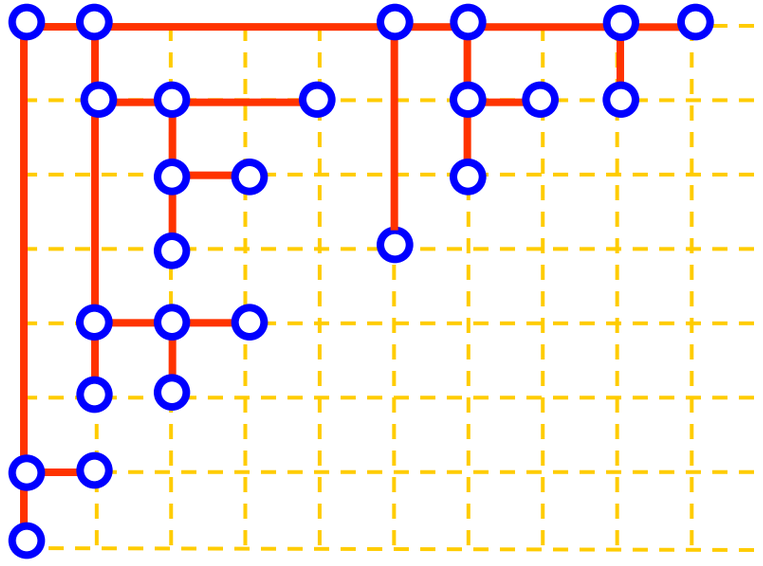
- Edges
  - are straight segments
  - do not intersect
- Nodes
  - have integer coordinates
  - are drawn either on the right or below their parent



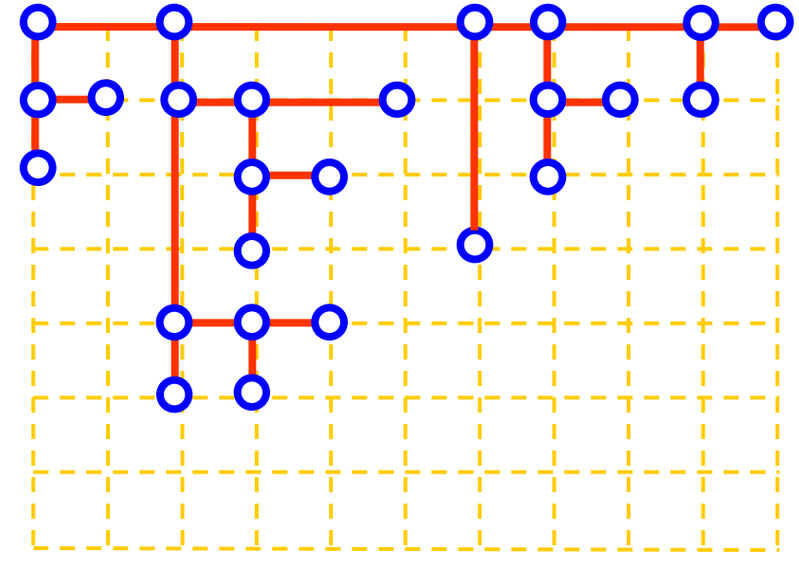
copyright ©2024 m.patrignani

Aesthetics for hv-drawings

- The purpose is to obtain a drawing with minimum area



area = 9 × 7 = 63

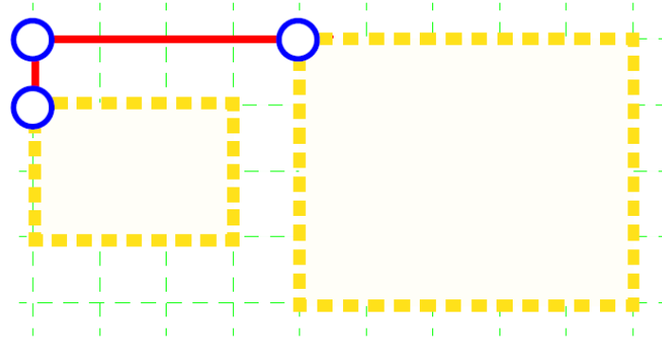


area = 10 × 5 = 50

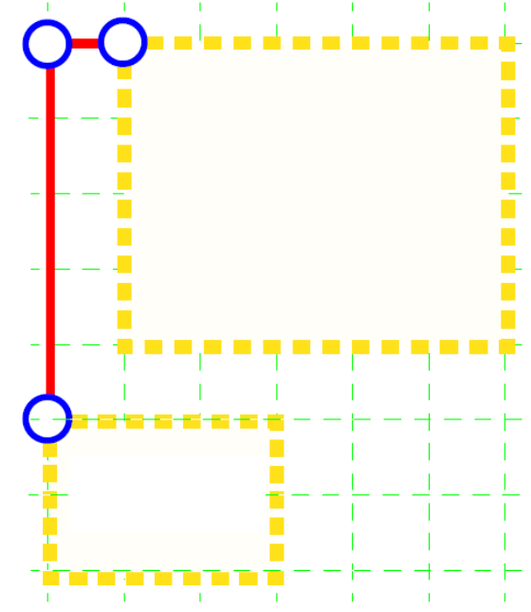
copyright ©2024 m.patrignani

hv combinations

- We can obtain a drawing by recursively combining the drawings of subtrees
  - two notable hv combinations



horizontal combination
the vertical segment has length one

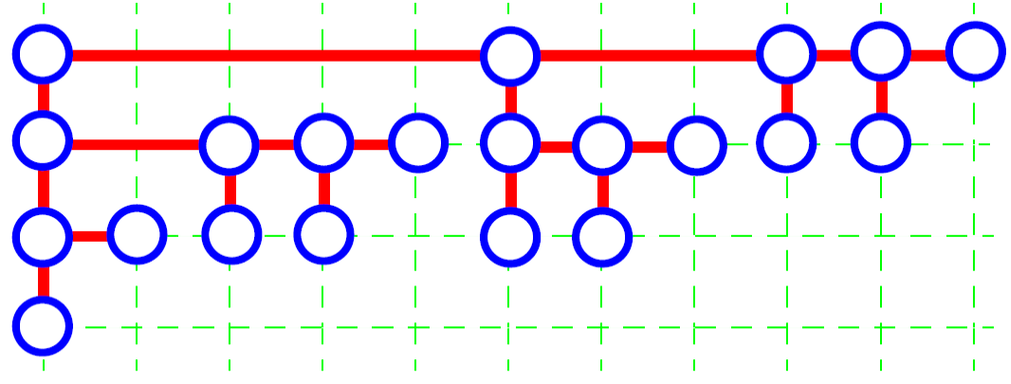


vertical combination
the horizontal segment has length one

copyright ©2024 m.patrignani

A very simple algorithm

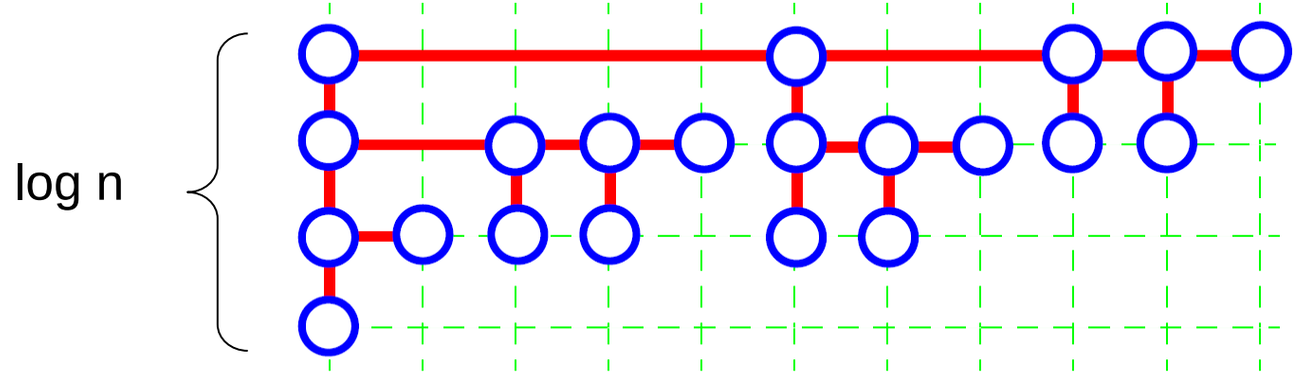
- Right heavy [Crescenzi *et al.*, 92]
  - only uses horizontal combinations
  - recursively places the largest subtree to the right
    - largest means that the subtree has more nodes
  - vertical segments have always length one



 copyright ©2024 m.patrignani

Bounding the height

- Each vertical edge (*u*,*v*) is such that the subtree rooted at the parent vertex *u* is at least twice the subtree rooted at *v*
  - hence the number of vertical edges on the left side of the drawing is at most log n



 copyright ©2024 m.patrignani

Bounding area and time

- What is the asymptotical area used by algorithm Right-heavy?
- What is the asymtotical time used by algorithm Right-heavy?

 copyright ©2024 m.patrignani

Properties of Right-heavy drawings

- The area is O(n log n)
  - the width is at most n-1
    - the width is n-1 if you only have a path
  - the height is at most log n
- Simply and axially isomorphic subtrees have congruent drawings, up to a translation
- Aspect ratio is poor Ω(n / log n)

 copyright ©2024 m.patrignani

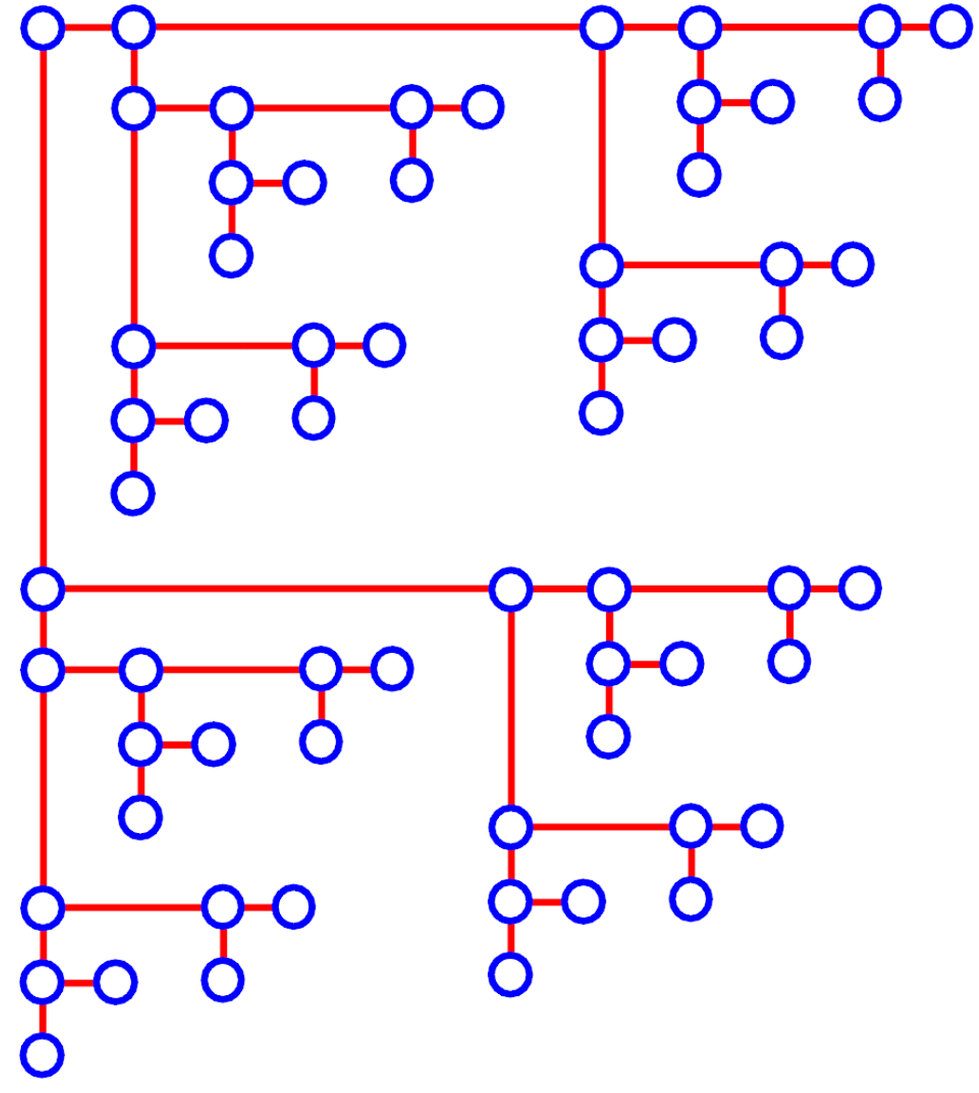
Improving aspect ratio

- In order to get a better aspect ratio, we should use both horizontal and vertical combinations
  - for example, given a complete binary tree, we can use horizontal combinations for subtrees rooted at vertices of odd depth, and vertical combinations for subtrees rooted at vertices of even depth

 copyright ©2024 m.patrignani

Drawing of a complete tree

- A drawing obtained alternating horizontal and vertical compositions
  - achieving O(n) area and constant aspect ratio for complete graphs



copyright ©2024 m.patrignani

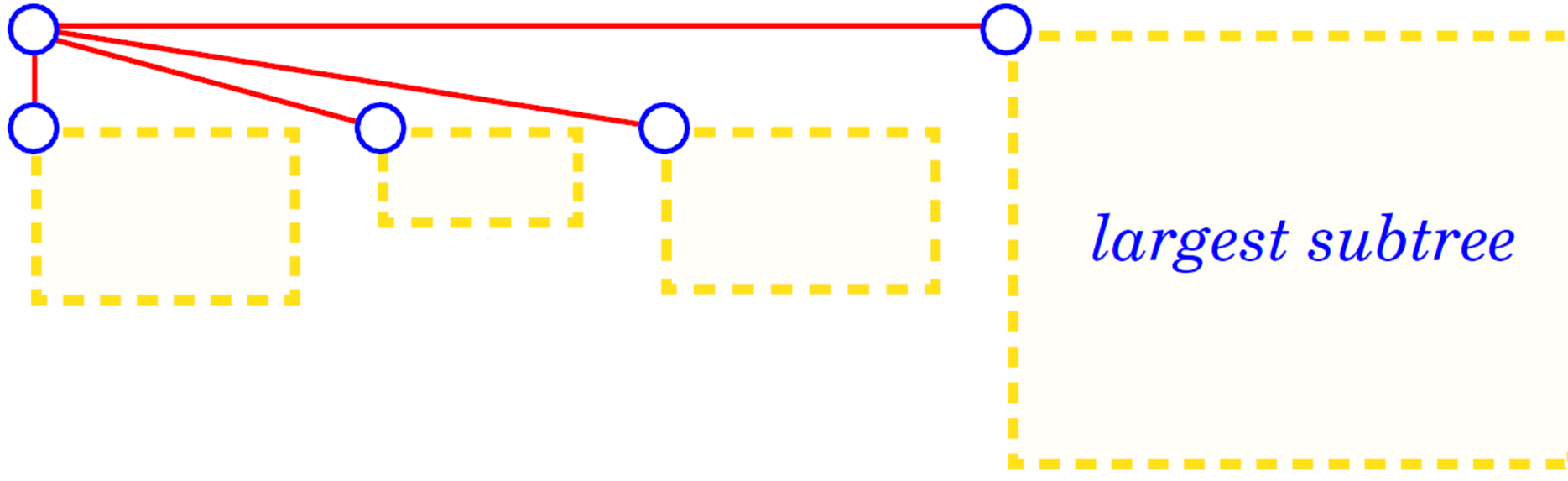
Improving area

- [Eades *et al.*, 92] show how to find an hv-drawing of a tree with optimal area in $O(n^2)$ time

 copyright ©2024 m.patrignani

hv-drawings for general trees

- The Right-heavy algorithm can be generalized to trees of arbitrary degree
  - same time and area bounds



 copyright ©2024 m.patrignani

Bibliography and further readings

- [Buchheim *et al.*, 06] C. Buchheim, M. Jünger, S. Leipert, “Drawing rooted trees in linear time”. Software-Practice and Experience 2006
- [Crescenzi *et al.*, 92] P. Crescenzi, G. Di Battista, A. Piperno, “A Note on Optimal Area Algorithms for Upward Drawings of Binary Trees”, Comput. Geom. Theory Appl., 1992
- [Di Battista *et al.* 99] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis. Graph Drawing. Prentice Hall, Upper Saddle River, NJ, 1999
- [Eades *et al.*, 92] P. Eades, T. Lin and X. Lin, “Minimum Size h-v Drawings”, Advanced Visual Interfaces, 1992
- [Knuth 71] Donald E. Knuth, “Optimum binary search trees”, Acta Informatica, 1971
- [Mill] Bill Mill, “Drawing Presentable Trees”, https://llimllib.github.io/pymag-trees/#foot3
- [Reingold & Tilford, 81] Edward M. Reingold and John S. Tilford, “Tidier drawings of trees” IEEE Trans. Soft. Eng. 1981
- [Supowit & Reingold, 82] Kenneth J. Supowit, Edward M. Reingold “The Complexity of Drawing Trees Nicely”. Acta Informatica, 1982
- [Walker 90] J. Q. II Walker, “A node-positioning algorithm for general trees”, Software-Practice and Experience 1990
- [Wetherell & Shannon 79] C. Wetherell, A. Shannon, “Tidy Drawings of Trees”, IEEE Transactions on Software Engineering. Vol. 5. 1979

 copyright ©2024 m.patrignani